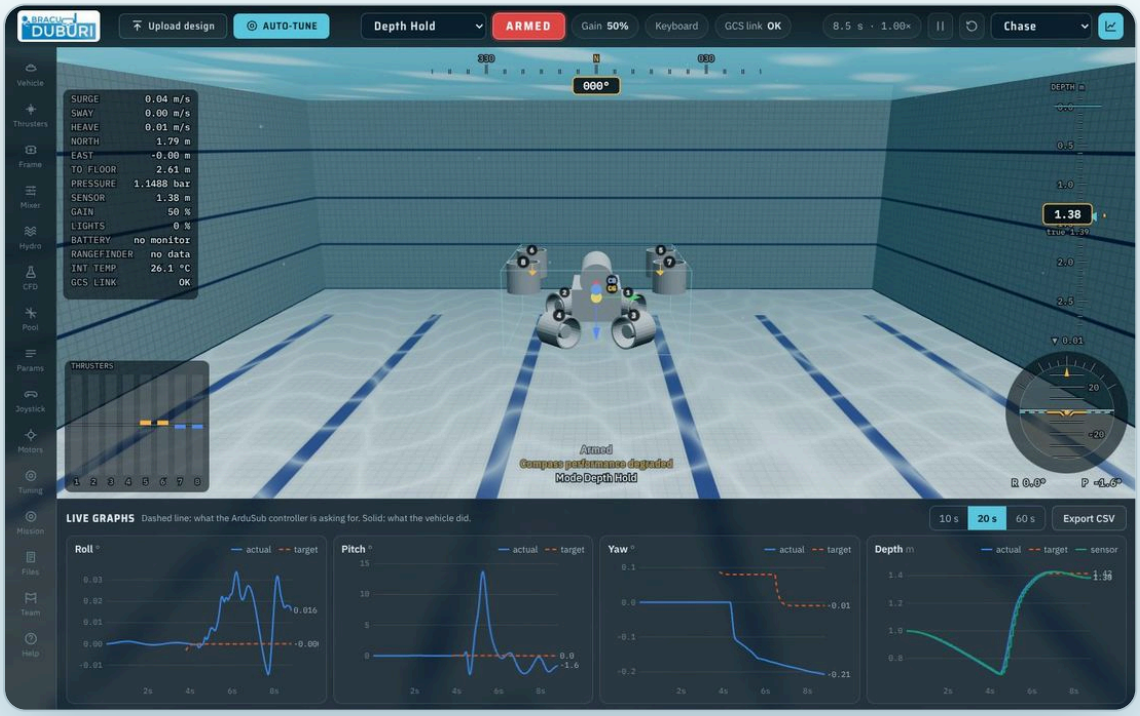
BRACU DUBURI · ROV SIMULATOR

Simulator Handbook

User guide, physics and ArduSub 4.5.7 control reference for the DUBURI ROV simulator.

Fossen 6-DOF | ArduSub 4.5.7 port | BEM · CFD | 2708 parameters | 24 screenshots



Version 1.0 · October 2026
jarifalam13.github.io/DUBURI_SIM

Open the simulator
jarifalam13.github.io/DUBURI_SIM

Contents

Every entry, cross-reference and web address in this PDF is a link. The bookmarks panel of your PDF reader lists every heading.

GUIDE

01 **Overview** 3
What the simulator is and what it gives you

02 **Quick start** 4
From your CAD file to a tuned .param file, in nine steps

03 **Panel tour** 9
Every panel of the simulator at a glance

04 **Flying and the HUD** 11
Keyboard, gamepad and every readout on screen

05 **Thruster auto-detection** 13
How thrusters are found in an uploaded design

06 **Flight modes** 14
The eleven ArduSub modes and what they need

07 **Failsafes** 15
What trips each failsafe and what it does

HOW IT WORKS

08 **Architecture** 16
The set-up pipeline and the 400 Hz loop

09 **The Fossen 6-DOF model** 17
Equations of motion, mass, Coriolis, damping, buoyancy, waves, contact, integration

10 **Thrusters and battery** 22
PWM to thrust, rotor dynamics, current, reaction torque, battery

11 **Added mass and damping** 24
Mass properties, panel method, lattice-Boltzmann CFD

12 **Sensors** 26
IMU, depth, estimator, compass, rangefinder and more

13 **ArduSub control** 28
PID, attitude, depth and position, mixer, main loop

14 **AUTO-TUNE** 31
How gains are suggested

REFERENCE

15 **Parameters** 32
How parameters load, and the 78 values the DUBURI file changes

16 **Accuracy and limits** 36
How close to the real vehicle, and what is not modelled

17 **Presenting the simulator** 37
Pitch, demo and likely questions

18 **Troubleshooting** 39
Common messages and fixes

19 **Development** 40
Build, test and source layout

CHAPTER 01 · GUIDE

01

Overview

What the simulator is and what it gives you

DUBURI is a virtual pool that runs in the browser. Give it your ROV's 3D design and your Pixhawk parameter file, and it flies the vehicle with a line-by-line port of the ArduSub 4.5.7 controllers on top of a Fossen 6-DOF hydrodynamic model.

400 Hz ArduSub main loop, physics at 800 Hz	**6-DOF** Fossen rigid body + added mass	**2708** ArduSub parameters, 671 simulated
11 modes Manual to Auto and Motor Detect	**185 tests** physics, firmware, params, CFD	**1 file** runs in Chrome or Edge, no install

Use it to check a thruster layout, catch wrong parameters, tune gains and practise piloting without risking the vehicle. Nothing is installed: the whole simulator is one HTML page (Chrome or Edge with WebGL2).

YOU GIVE IT	IT WORKS OUT	YOU GET BACK
3D design (STL, OBJ, PLY)	Volume, buoyancy, centre of buoyancy, inertia, thruster positions	Whether it floats level, which axes it can control
Mass and centre of gravity	Righting moment, natural roll and pitch periods	Stability before you build
Thruster models and directions	Mixer table, authority per axis, battery draw	A layout check and a C++ mixer for custom frames
Pixhawk `.param` file	The same controllers ArduSub runs, with your gains	Step responses, AUTO-TUNE suggestions, an updated `.param`
Pool size, water, waves	Pressure, density, depth-sensor reading, wave forces	Realistic depth hold and failsafe behaviour

CHAPTER 02 · GUIDE

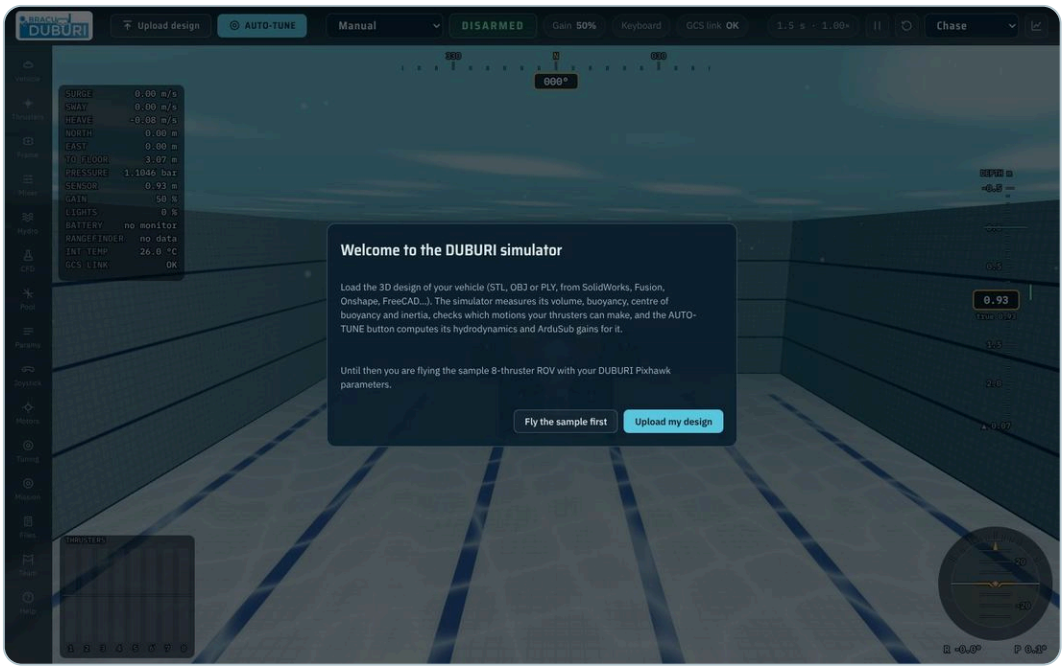
02

Quick start

Nine steps from your CAD file to a tuned `.param` file.

1 Open the simulator

Open jarifalam13.github.io/DUBURI_SIM in Chrome or Edge. The first screen offers **Fly the sample first** (an 8-thruster ROV already loaded with the DUBURI Pixhawk parameters) or **Upload my design**.

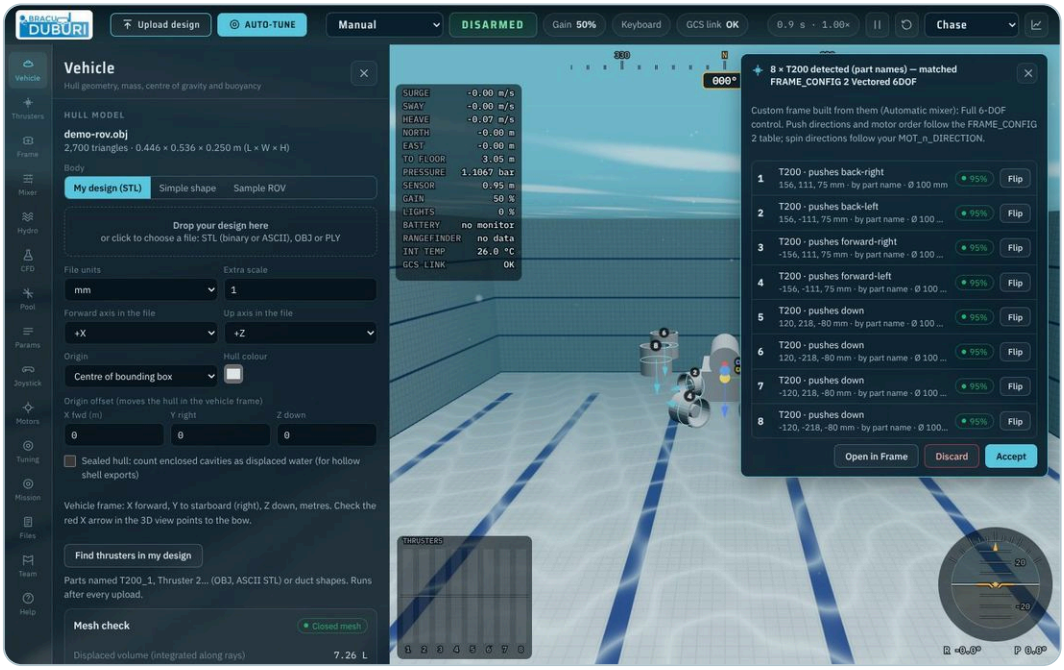


Welcome. The sample ROV flies with your DUBURI parameter file from the start.

2 Upload your design

Click **Upload design** in the top bar, or drop the file anywhere. In **Vehicle**, set the file units (Fusion 360 and SolidWorks export in mm), which axis points forward and which points up. Check that the red X arrow in the 3D view points to the bow.

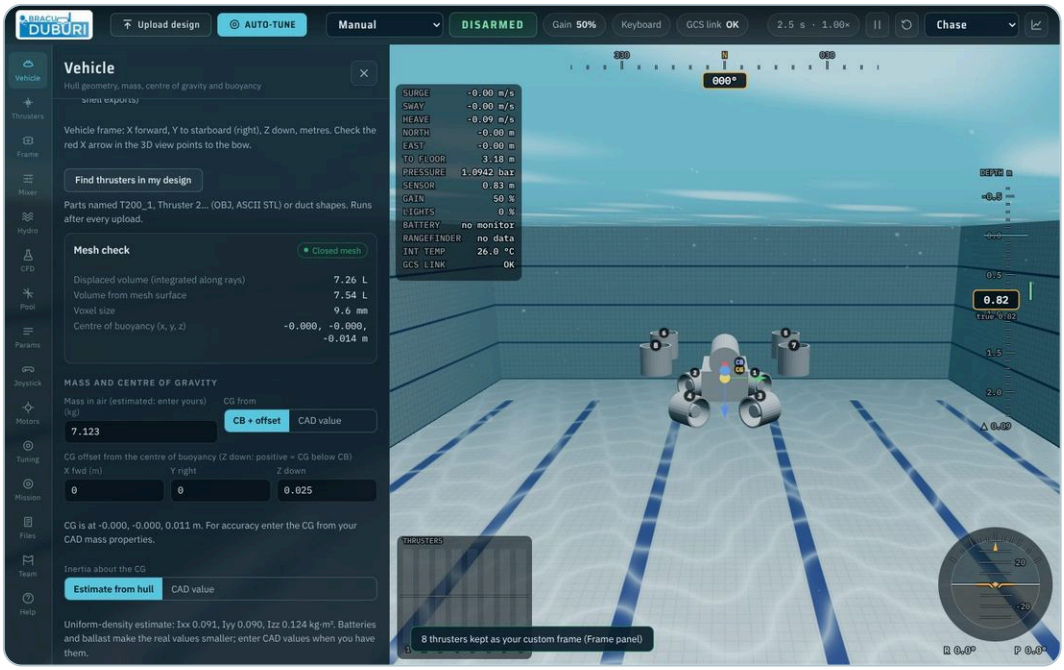
If the parts are named (`T200_1` , `Thruster 2` ...) or the thrusters have ducts, they are found automatically and a custom frame is built from them. Click **Accept**, **Flip** a thruster that pushes the wrong way, or **Discard**.



Thruster detection. 8 × T200 found by part name and matched to FRAME_CONFIG 2 (Vectored 6DOF), each with its push direction and confidence.

3 Enter the real mass and centre of gravity

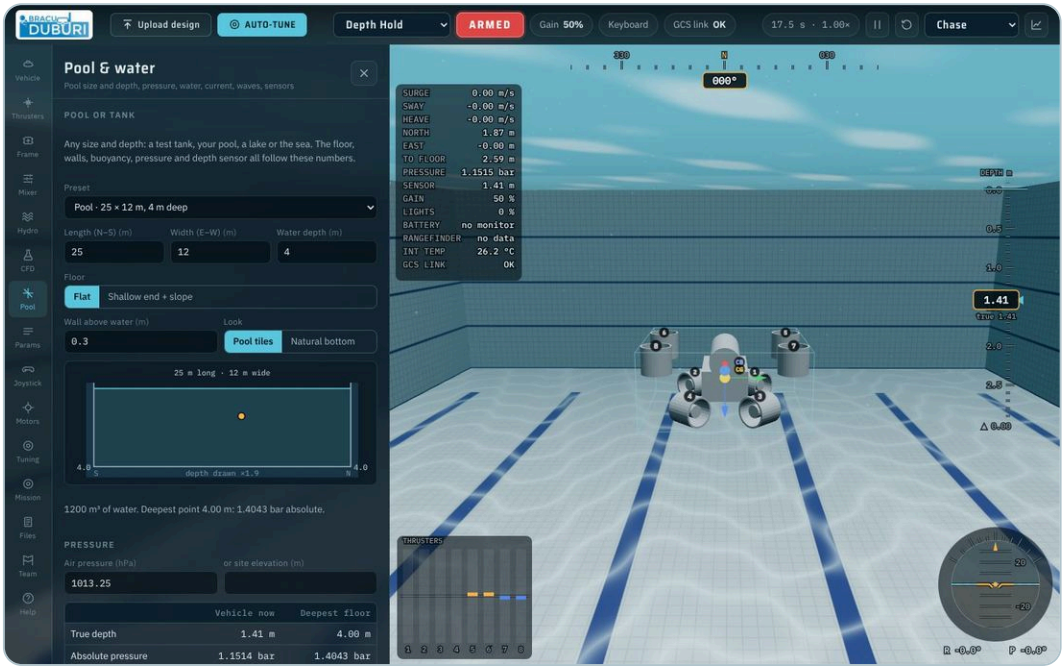
Scroll down in **Vehicle** to **Mass and centre of gravity**. Enter the mass in air from a scale and the CG from your CAD mass properties. The **In the water** box then shows whether the vehicle floats or sinks, the distance between centre of buoyancy and CG (GM), and the natural roll and pitch periods. **Make neutral** sets the mass for neutral buoyancy.



Mass, CG and buoyancy. Every hydrostatic number the plant uses is shown here.

4 Set the pool and water

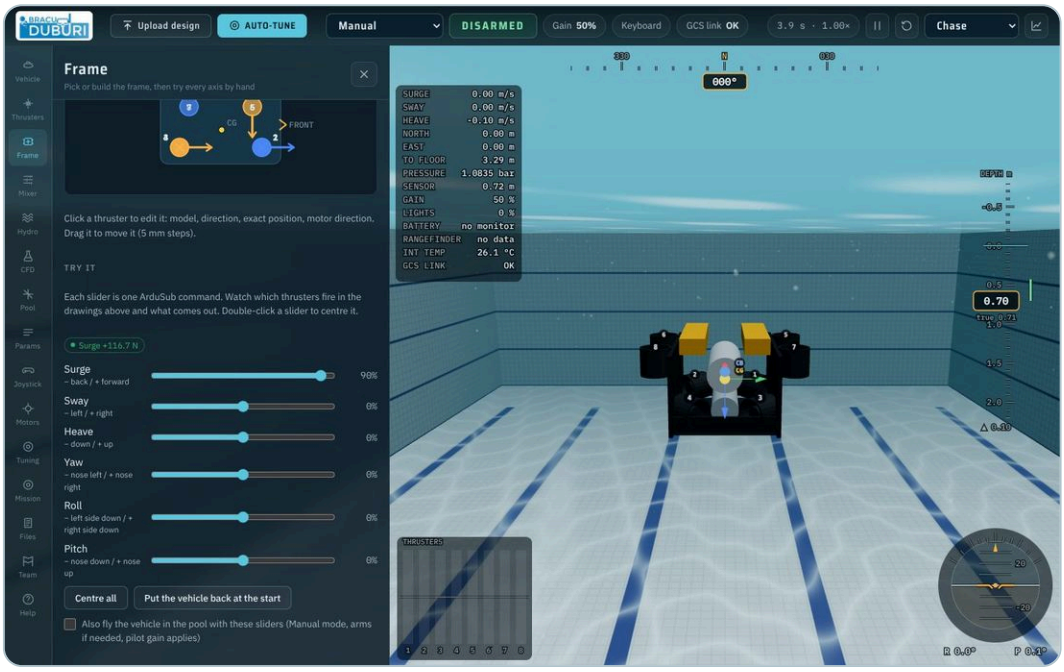
Pool sets the pool length, width and depth (default 25 × 12 × 4 m), a flat or sloped floor, fresh or sea water, current, turbulence and waves, the depth sensor (Bar02, Bar30 or custom) and the depth rating of your housings. Tick **Positioning system present** only if the real vehicle has a DVL or USBL: Position Hold, Auto and Guided need it.



Pool & water. Pressure, buoyancy and the depth-sensor reading all follow from these settings.

5 Check the frame

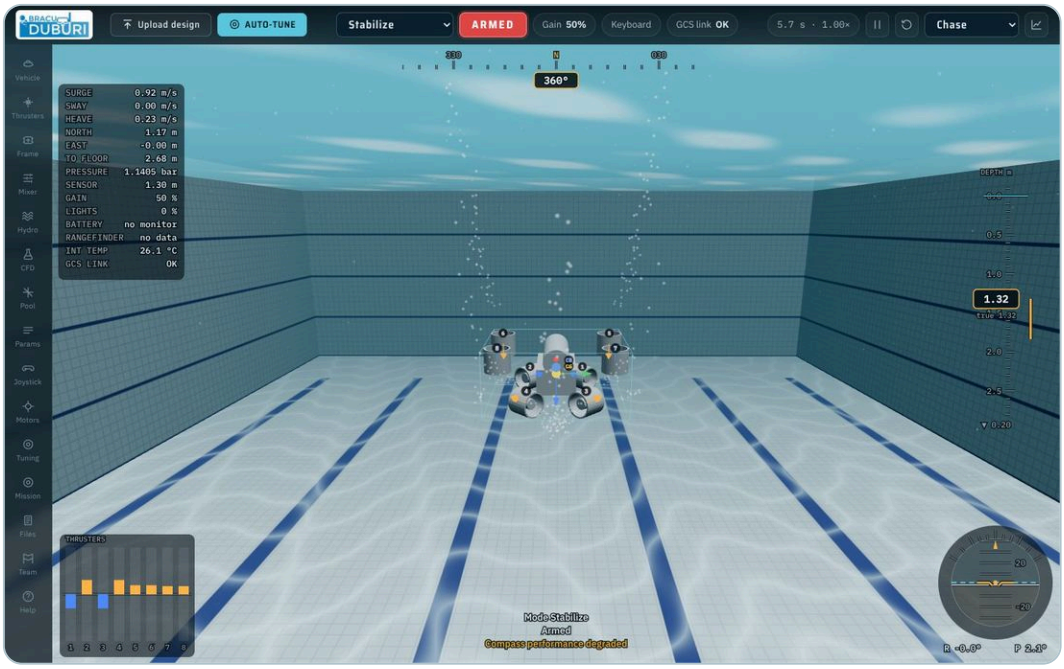
Frame shows every ArduSub `FRAME_CONFIG` as a card, plus **Automatic**, which builds the mixer from where your thrusters really are. Under **Try it**, move a slider: the thrusters that fire light up (orange forward, blue reverse) and the resulting force or torque is printed above the sliders.



Try it. Surge at 90 % gives +116.7 N with this layout. A slider that moves the vehicle the wrong way means a direction or `MOT_n_DIRECTION` is wrong.

6 Arm and fly

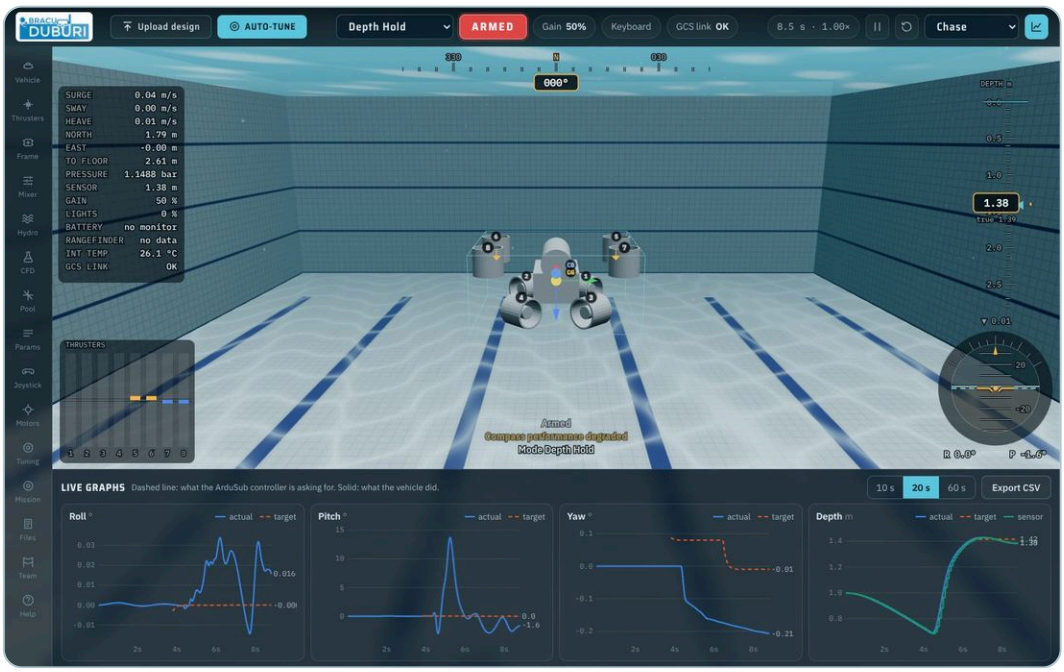
Press `Enter` to arm. `W` / `S` forward and back, `A` / `D` yaw, `Q` / `E` strafe, `R` / `F` up and down. Keys `1` – `6` change mode. A gamepad works too (see Flying).



Flying. Heading tape at the top, depth tape on the right, thruster bars bottom left, attitude bottom right.

7 Hold depth and watch the graphs

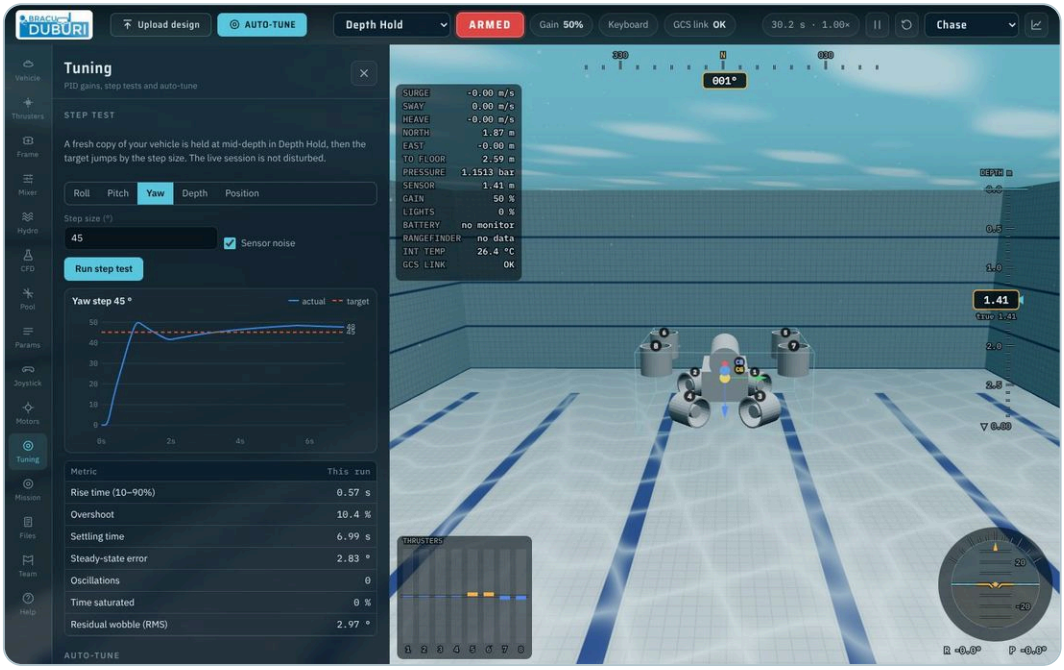
Press `3` for Depth Hold and `G` for the live graphs. Dashed lines are what ArduSub asks for, solid lines what the vehicle did. **Export CSV** saves the data.



Live graphs. The depth plot shows target, true depth and what the depth sensor reports.

8 Tune

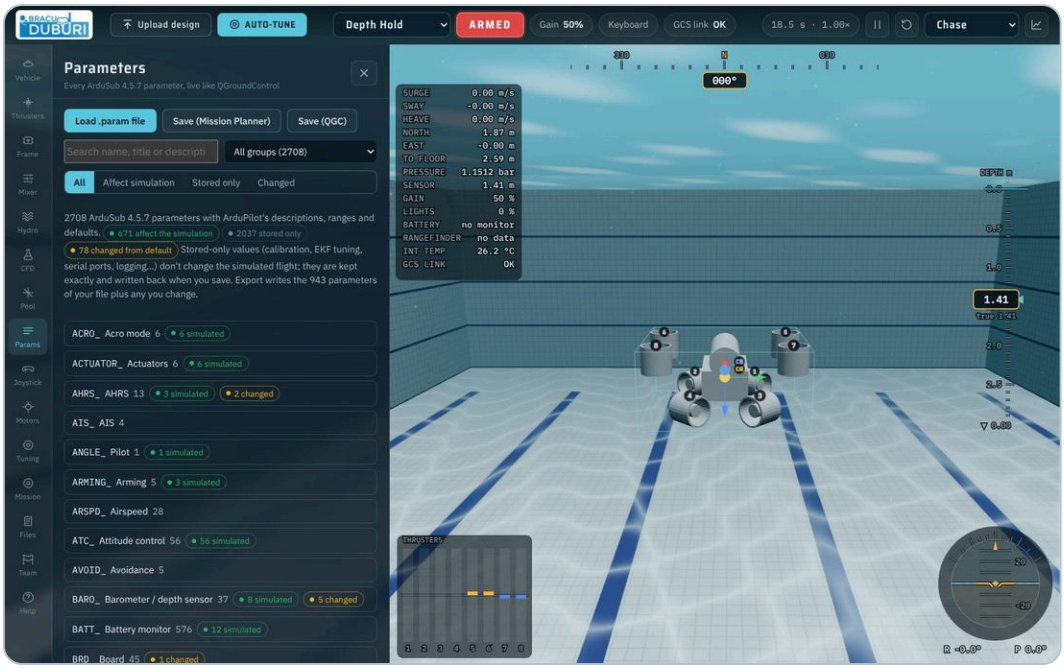
Tuning lists the ArduSub gains and runs a **step test** on a fresh copy of the vehicle: rise time, overshoot, settling time and steady error are measured. **AUTO-TUNE** (top bar) computes added mass and damping for your design and suggests gains; nothing changes until you click Apply.



Step test. A 45° yaw step: 0.57 s rise time, 10.4 % overshoot.

9 Take it to the vehicle

In **Params**, **Save (Mission Planner)** or **Save (QGC)** writes every parameter of your file plus the ones you changed. Load it on the Pixhawk with QGroundControl or Mission Planner and check the changed values before the first dive.



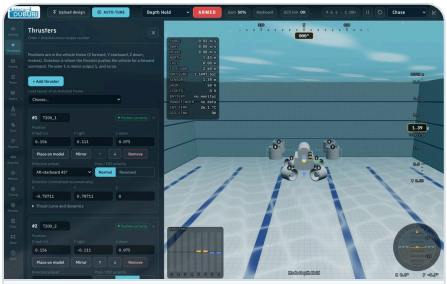
Parameters. 2708 ArduSub 4.5.7 parameters: 671 affect the simulation, the rest are kept exactly and written back.

CHAPTER 03 · GUIDE

03

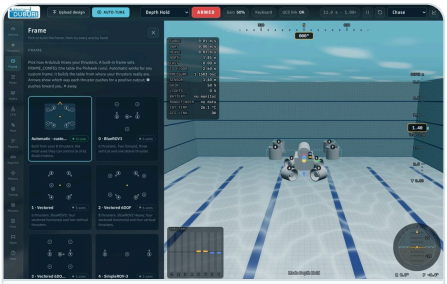
Panel tour

The rail on the left of the simulator opens these panels, in the order you usually need them.



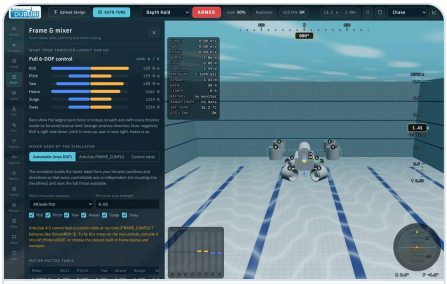
Thrusters

Order = ArduSub motor output. Position, direction, polarity, model (T200, T500, generic) or your measured PWM-to-thrust table.



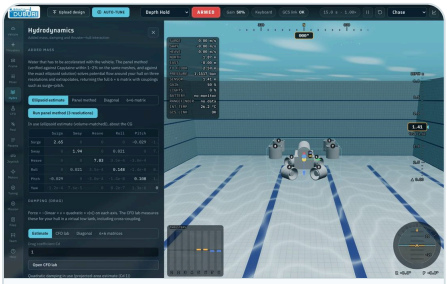
Frame

Every FRAME_CONFIG as a card or Automatic; custom frame builder; Try it sliders; copy the mixer as add_motor_raw_6dof() lines.



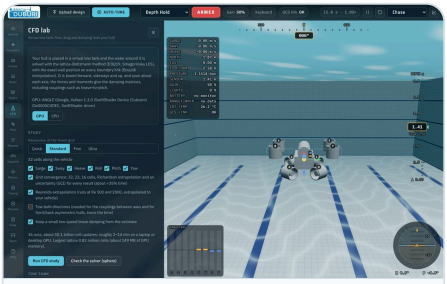
Frame & mixer

Which axes the layout controls, authority and cross-coupling per axis, saturation scheme.



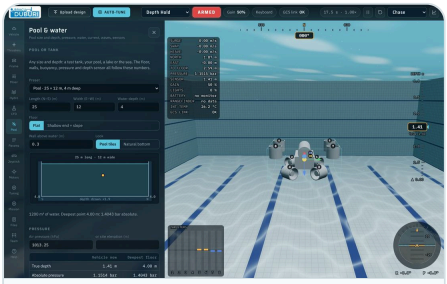
Hydrodynamics

Added-mass and damping matrices, their source and uncertainty, thruster–hull interaction.



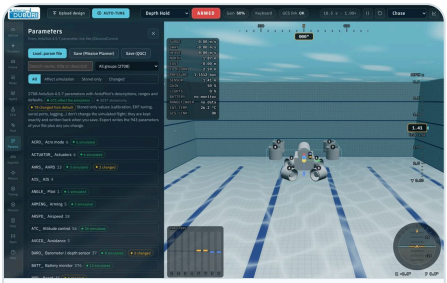
CFD lab

Virtual tow tank on the GPU: flow, drag and damping from your hull with grid convergence.



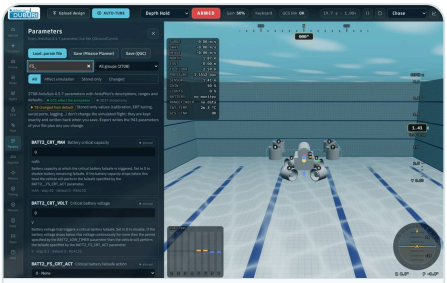
Pool & water

Pool size and floor, water, current, waves, pressure, depth sensor and positioning.



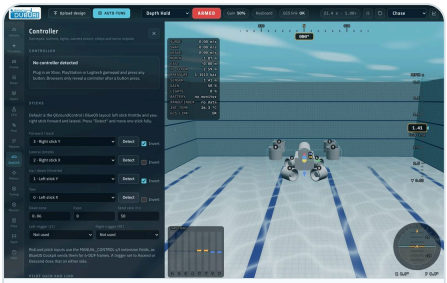
Parameters

All 2708 ArduSub parameters, searchable, with ranges, value lists and bitmasks; load and save .param.



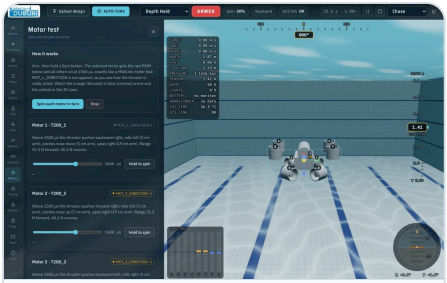
Parameter search

Search by name or words; filter to simulated, stored-only or changed values.



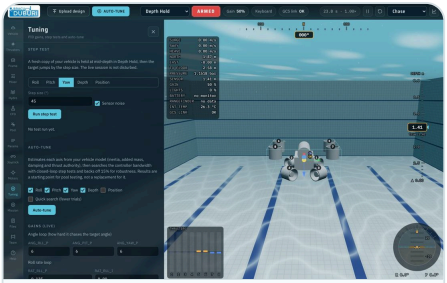
Controller

Gamepad buttons and shift functions, lights, camera mount, relays, servo outputs, RC1–16.



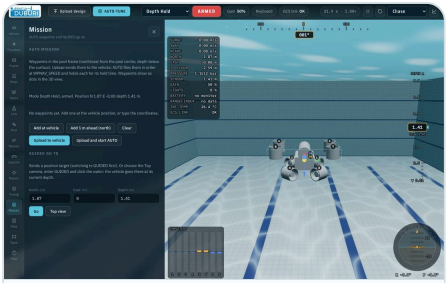
Motor test

Spin one thruster at a time, as MAV_CMD_DO_MOTOR_TEST does.



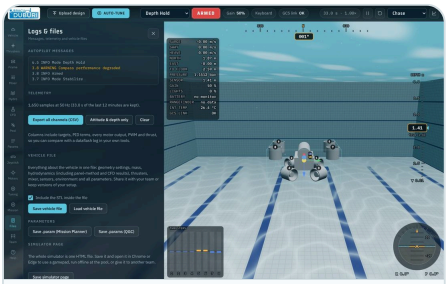
Tuning

ArduSub gains grouped by loop, step tests with metrics.



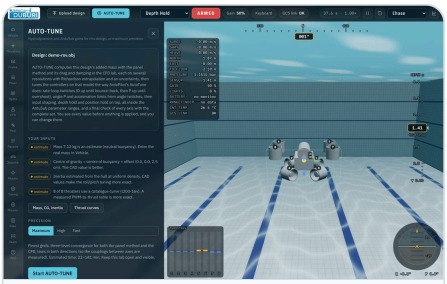
Mission

AUTO waypoints (north, east, depth, hold) and GUIDED go-to.



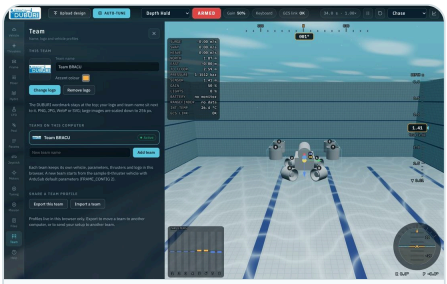
Logs & files

Autopilot messages, telemetry CSV, vehicle files, save the page for offline use.



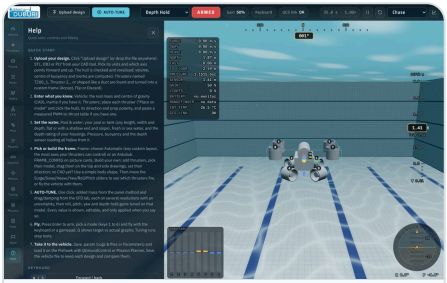
AUTO-TUNE

Panel-method added mass, CFD damping, then ArduPilot-style twitch tuning; suggestions only.



Team

Team name, logo and colour; each team keeps its own vehicles in the browser.



Help

Quick start, keyboard and gamepad, conventions, what is simulated and the limits.

CHAPTER 04 · GUIDE

04

Flying and the HUD

Flight keys work whenever no text field has the focus.

Keyboard

KEY	ACTION
`W` `S`	Forward / back
`A` `D`	Yaw left / right
`Q` `E`	Strafe left / right
`R` `F` (`Space` `C`)	Ascend / descend
`I` `K` · `J` `L`	Pitch · roll input (vehicles with 6-DOF control)
`Enter`	Arm / disarm
`1` … `6`	Manual, Stabilize, Depth Hold, Acro, Position Hold, Surface (other modes from the mode menu)
`+` `−`	Pilot gain up / down (JS_GAIN_STEPS)
`V`	Next camera (Chase, Orbit, Camera, Top, Free)
`G`	Live graphs
`P`	Pause
`Backspace`	Reset the vehicle to its start position
`Shift`	Presses the joystick button set to shift (BTNn_FUNCTION = 1)

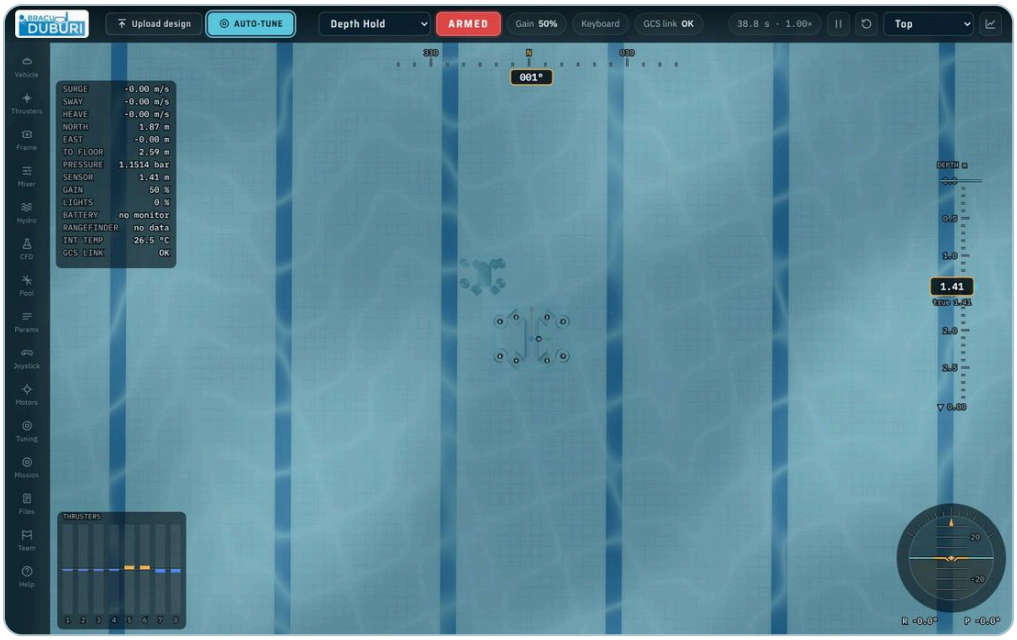
Gamepad

CONTROL	ACTION
Left stick	Up / down (throttle) and yaw
Right stick	Forward and lateral
Triggers	Ascend / descend, or pitch / roll when the matching button function is held
Buttons	Follow your `BTNn_FUNCTION` / `BTNn_SFUNCTION` parameters exactly as ArduSub does (QGC button numbering); hold the shift button for the second function

The Controller panel lists every button, its function with and without shift, lights, camera mount, relays, servo outputs and RC1–16. If the gamepad is not detected, press any button once; some embedded browser views hide controllers.

HUD readouts

READOUT	MEANING
SURGE · SWAY · HEAVE	Body-frame velocity, m/s (forward, right, down)
NORTH · EAST	Position in the pool, m from the centre
TO FLOOR	Distance from the vehicle to the pool floor, m
PRESSURE · SENSOR	Absolute pressure at the depth sensor, bar, and the depth ArduSub computes from it, m
Depth tape	ArduSub's depth estimate in the box, with the true CG depth under it and the target depth marker
GAIN · LIGHTS	Pilot gain and light brightness, %
BATTERY	Voltage, current, mAh used, % left (shows "no monitor" when `BATT_MONITOR` = 0)
RANGEFINDER · INT TEMP	Downward rangefinder distance, internal temperature
GCS LINK	Ground-station link state; the topbar toggle drops it to test the GCS failsafe
THRUSTERS	One bar per motor output: orange forward, blue reverse
Banners	Active failsafes and PreArm failures, as ArduSub reports them



Top view. In Guided mode a click on the water sends the vehicle there at its current depth.

CHAPTER 05 · GUIDE

05

Thruster auto-detection

Runs after every upload, and from **Find thrusters in my design** in Vehicle or Frame.

FORMAT	PART NAMES KEPT	HOW THRUSTERS ARE FOUND
OBJ (recommended)	yes objects and groups	By name, then by shape
ASCII STL, multi-body	yes one `solid` per body	By name, then by shape
Binary STL	no	By shape only
PLY	no	By shape only

1. **Names.** Parts called `T200_1`, `Thruster 2`, `Assembly/T200 (4)/Nozzle` are recognised; the number becomes the motor number. Mounts, brackets and screws are skipped.
2. **Shape.** The mesh is split into connected parts; each is tested as a surface of revolution. A duct needs a complete outer and inner wall. Outside diameter ≈ 100 mm → T200, ≈ 140 mm → T500, otherwise a generic thruster sized from the inside diameter. Pipes, rods and housings are rejected.
3. **Direction and order.** Geometry gives the axis but not which way it pushes, so each thruster's force and torque is matched against the rows of your `FRAME_CONFIG` table (with an optimal one-to-one assignment). That picks the push direction and the motor number. Spin direction follows `MOT_n_DIRECTION`.

From Fusion 360 (.f3z)

The browser cannot read Fusion archives. Open the design in Fusion, rename the thruster components `T200_1` … `T200_8` to match your motor outputs, then **File → Export → OBJ** (or **Utilities → 3D Print** → OBJ). Upload with units = mm.

Typical designs take 50–90 ms; a 370 000-triangle mesh about 0.45 s. A short hollow housing 85–115 mm across can look like a T200: check the list and discard it.

CHAPTER 06 · GUIDE

06

Flight modes

The same entry rules as ArduSub 4.5.7 (`ArduSub/mode.h`): a mode that needs position is refused without a position estimate, and leaving a manual-throttle mode needs a working depth sensor.

KEY	MODE	NO.	POSITION	DEPTH SENSOR	WHAT IT DOES
1	**Manual**	19	not needed	not needed	Pilot commands go straight to the mixer. No stabilisation.
2	**Stabilize**	0	not needed	not needed	Keeps roll and pitch level and holds heading when the yaw stick is centred. The pilot controls up and down.
4	**Acro**	1	not needed	not needed	Sticks command rotation rates in the body frame. No self-levelling.
3	**Depth Hold**	2	not needed	needed	Stabilize plus depth hold from the depth sensor. The throttle stick changes depth at PILOT_SPEED_UP / PILOT_SPEED_DN.
menu	**SurfTrak**	21	not needed	needed	Depth hold that keeps a constant distance above the bottom with the rangefinder, once deeper than SURFTRAK_DEPTH.
5	**Position Hold**	16	needed	needed	Holds position, depth and heading. Needs a DVL or USBL.
6	**Surface**	9	needed	needed	Climbs to the surface, then switches to Depth Hold.
menu	**Guided**	4	needed	needed	Goes to a target position: Mission panel, or a click in Top view.
menu	**Auto**	3	needed	needed	Flies the waypoint mission (WPNAV_* speeds) and loiters at the end.
menu	**Circle**	7	needed	needed	Circles a centre point at CIRCLE_RADIUS and CIRCLE_RATE.
menu	**Motor Detect**	20	not needed	needed	Pulses each motor and sets MOT_n_DIRECTION from how the vehicle responds.

Surface needs a position estimate in ArduSub 4.5.7, so a leak or GCS failsafe set to "surface" only warns on a vehicle without DVL/USBL. The simulator reproduces this.

CHAPTER 07 · GUIDE

07

Failsafes

Ported from `ArduSub/failsafe.cpp`. The last column is the value in the DUBURI Pixhawk file.

FAILSAFE	TRIGGERS WHEN	PARAMETER AND ACTIONS	ARDUSUB DEFAULT	DUBURI FILE
Pilot input	No joystick message for `FS_PILOT_TIMEOUT` (3 s)	`FS_PILOT_INPUT`: 0 off · 1 warn · 2 disarm	2 disarm	0 off
GCS heartbeat	No ground-station heartbeat for 2.5 s	`FS_GCS_ENABLE`: 0 off · 1 warn · 2 disarm · 3 depth hold · 4 surface	2 disarm	0 off
Leak	A leak probe reads wet (housing past its rating)	`FS_LEAK_ENABLE`: 0 off · 1 warn · 2 surface	1 warn	0 off
Internal pressure	Pressure inside the housing above `FS_PRESS_MAX` (105 000 Pa)	`FS_PRESS_ENABLE`: 0 off · 1 warn	0 off	0 off
Internal temperature	Above `FS_TEMP_MAX` (62 °C)	`FS_TEMP_ENABLE`: 0 off · 1 warn	0 off	0 off
Battery	Voltage below `BATT_LOW_VOLT` for `BATT_LOW_TIMER` (10 s), or mAh used past `BATT_LOW_MAH`; same for critical	`BATT_FS_LOW_ACT`, `BATT_FS_CRT_ACT`: 0 none · 2 disarm · 3 surface	0 none	no monitor (`BATT_MONITOR` 0)
EKF	Estimator variance stays above `FS_EKF_THRESH` (0.8)	`FS_EKF_ACTION`: 0 off · 1 warn · 2 disarm	0 off	0 off
Crash	Attitude error above 30° for 2 s	`FS_CRASH_CHECK`: 0 off · 1 warn · 2 disarm	0 off	0 off
Depth sensor	The depth sensor stops reporting while a depth mode is active	Always on: switches to Manual	on	on
Terrain	An Auto/Guided target that follows the bottom with no healthy rangefinder for 1 s	`FS_TERRAIN_ENAB`: 0 disarm · 1 hold position · 2 surface	0 disarm	0 disarm

The DUBURI file turns off three failsafes ArduSub enables by default

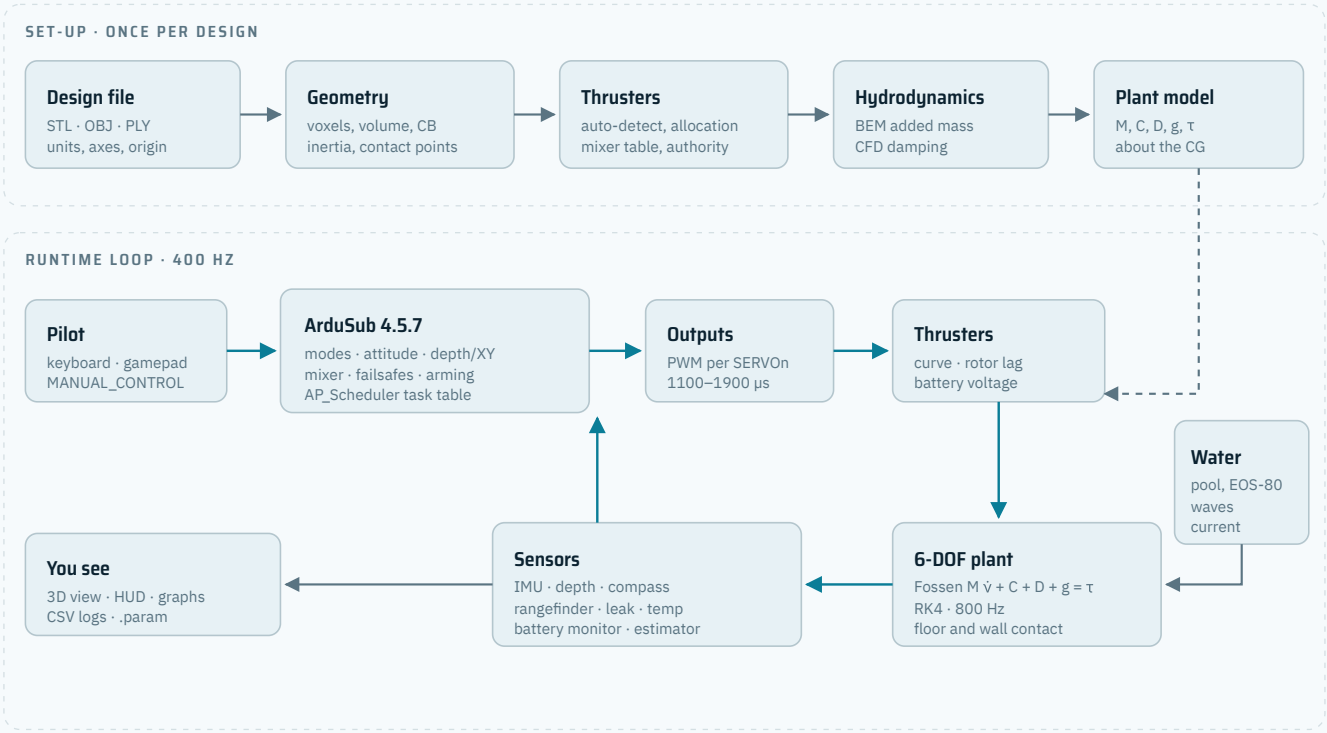
Pilot input (default disarm), GCS heartbeat (default disarm) and leak (default warn) are all 0. Try each one in the simulator first: set the parameter in Params, then drop the joystick or click **GCS link** in the top bar. With `FS_GCS_ENABLE` = 3 the vehicle switches to Depth Hold about 2.5 s after the link drops.

CHAPTER 08 · HOW IT WORKS

08

Architecture

Two parts: a set-up pipeline that turns your design into a hydrodynamic model once, and a closed loop that runs 400 times per simulated second.



Design file. Your CAD export, read in the browser. Units, forward and up axes and the origin are set in Vehicle.

Thrusters and allocation. Thrusters are found by part name or duct shape, then matched to the FRAME_CONFIG table. The allocator finds which axes the layout controls and builds the mixer.

Plant model. All matrices moved to the CG, checked for symmetry and positive definiteness, plus thruster, battery and hull data.

ArduSub 4.5.7 firmware. The control code ported line by line: modes, attitude and depth controllers, AP_Motors6DOF mixer, failsafes, arming checks and the AP_Scheduler task table.

Thrusters. PWM to thrust from the T200/T500 table (mirrored for reversed motors), rotor-speed lag, battery voltage, current draw, reaction torque, in-flow and ventilation losses.

Water and pool. Pool geometry, density and viscosity from temperature and salinity, compressible pressure (EOS-80), finite-depth waves, current and turbulence.

What you see. The 3D pool, the Cockpit-style HUD, live target-vs-actual graphs, CSV telemetry at 50 Hz and the exported .param file.

Geometry analysis. The hull is voxelised. Volume and centre of buoyancy are integrated exactly along rays; inertia about the CG follows from the mass you enter.

Hydrodynamic coefficients. Added mass from a panel method (BEM) and damping from lattice-Boltzmann CFD, each on several grids with an uncertainty. Run by AUTO-TUNE or the CFD lab.

Pilot input. Keyboard or gamepad, sent as MAVLink MANUAL_CONTROL at 50 Hz with a 1 Hz heartbeat, exactly like QGroundControl.

Servo outputs. Motor m drives the output whose SERVOn_FUNCTION is 33 + m. The ESC sees the previous tick's value, latched at RC_SPEED for PWM.

6-DOF plant. Fossen's equations: rigid body plus added mass, Coriolis and Munk terms, coupled damping, buoyancy at the CB, Froude–Krylov forces, floor and wall contact. RK4 with a renormalised quaternion.

Sensors. What the firmware is allowed to know: IMU with noise and bias drift, depth averaged at 10 Hz, compass, rangefinder, leak probes, temperature, battery monitor and a vertical Kalman filter.

CHAPTER 09 · HOW IT WORKS

09

The Fossen 6-DOF model

The equations of motion used in marine robotics, from Thor I. Fossen, *Handbook of Marine Craft Hydrodynamics and Motion Control* (Wiley, 2011/2021). Implemented in `src/physics/body.js`.

A rigid body in water can move in six ways at once: three translations and three rotations. Those are the six **degrees of freedom** (DOF). Fossen's model is Newton's second law written for all six together, with the extra forces water adds: the water the vehicle carries along when it accelerates, the drag, and buoyancy.

DOF	MOTION	VELOCITY	FORCE / MOMENT	POSITION / ANGLE
1	Surge (forward)	u	X	x north
2	Sway (right)	v	Y	y east
3	Heave (down)	w	Z	z depth
4	Roll	p	K	ϕ
5	Pitch	q	M	θ
6	Yaw	r	N	ψ

Frames: world North-East-Down with the origin at the still-water surface in the middle of the pool; body Forward-Right-Down with the origin at the centre of gravity (CG). Positive roll is right side down, positive pitch nose up, positive yaw nose right (ArduPilot convention).

$$\underbrace{M\,\dot{\nu}}_{\text{inertia}} + \underbrace{C(\nu_r)\,\nu_r}_{\text{Coriolis, Munk}} + \underbrace{D(\nu_r)\,\nu_r}_{\text{drag}} + \underbrace{g(\eta)}_{\text{weight, buoyancy}} = \underbrace{\tau}_{\text{thrusters, contact, waves}} \qquad \dot{\eta} = J(\eta)\,\nu$$

η	position and attitude in the world frame (here a position and a unit quaternion)
ν	body velocities $[u, v, w, p, q, r]$; ν_r is the velocity relative to the water
M	6×6 inertia: the vehicle's own mass and inertia plus the **added mass** of water it pushes around
C	forces that appear when the vehicle moves and turns at the same time, including the Munk moment that turns a moving body broadside
D	water resistance, a linear part (slow speeds) and a quadratic part ($\propto$ speed²)
g	weight at the CG and buoyancy at the centre of buoyancy (CB): their offset is the righting moment

The code solves one linear system per evaluation. Everything except the inertia term is moved to the right-hand side:

$$(M_{RB} + M_F + M_{A,e})\dot{\nu} = \tau_{\text{grav}} + \tau_B + \tau_{FK} + \tau_D - C_{RB}(\nu)\nu - C_F(\nu)\nu - C_A(\nu_r)\nu_r + \tau_{\text{thr}} + \tau_c$$

$M_F,\ C_F$	water that has flooded a leaking housing, a point mass at the CB
τ_{FK}	Froude–Krylov force from accelerating water (waves, gusts)
τ_c	contact with the pool floor and walls

`body.js:1-4, 223-442 · the 6×6 matrix is inverted by Gauss–Jordan with partial pivoting (body.js:25-43)`

State vector, 13 numbers: $y = [x, y, z,\ q_w, q_x, q_y, q_z,\ u, v, w,\ p, q, r]$.

Kinematics: from body velocity to motion in the pool

Position changes with the body velocity rotated into the world frame. Attitude is a unit quaternion (no gimbal lock), turned into roll, pitch and yaw only for display, with ArduPilot's 3-2-1 order.

$$\dot{p}^n = R(q)\begin{bmatrix}u\\v\\w\end{bmatrix}, \qquad \dot{q} = \tfrac{1}{2}\, q \otimes \begin{bmatrix}0\\ \omega\end{bmatrix}, \qquad \omega = [p, q, r]^\top$$

$$R(q) = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1-2(q_x^2+q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1-2(q_x^2+q_y^2) \end{bmatrix}$$

`core/math.js:140-148, body.js:444-450`

Water velocity (current, turbulence, wave orbits) is rotated into the body frame, and only the linear part is made relative; the water is not rotating.

$$v_c^b = R^\top v_f^n(x,y,z), \qquad \nu_r = [\,u-u_c,\; v-v_c,\; w-w_c,\; p,\; q,\; r\,]^\top$$

`body.js:232-236`

Mass and added mass

Because the body origin is the CG, the rigid-body inertia is block-diagonal. When the vehicle accelerates it also has to accelerate some water: that is the **added mass** M_A, a full 6×6 matrix computed by the panel method (see Added mass and damping). It shrinks with the submerged fraction σ at the surface and grows near the floor, walls and surface (image effect).

$$M_{RB} = \begin{bmatrix} m I_3 & 0 \\ 0 & I_g \end{bmatrix}, \qquad M_{A,e} = \sigma\, S_f\, M_A\, S_f, \qquad S_f = \operatorname{diag}\!\left(\sqrt{f_1}, \sqrt{f_2}, \sqrt{f_3}, 1, 1, 1\right)$$

$$f_i \leftarrow f_i \left(1 + \begin{cases} \tfrac{3}{8}\, r^3 & i = \text{axis normal to the nearby plane} \\ \tfrac{3}{16}\, r^3 & \text{other axes} \end{cases}\right), \qquad r = \frac{a_{eq}}{\max(h, a_{eq})}, \quad a_{eq} = \sqrt[3]{\tfrac{3V}{4\pi}}$$

$$M_F = m_f \begin{bmatrix} I_3 & -S(r_b) \\ S(r_b) & -S(r_b)^2 \end{bmatrix}, \qquad m_f = \rho\, V_{\text{flood}}$$

h	distance from the CG to the floor, the surface (treated as a rigid lid) or a wall
$S(a)$	skew matrix, $S(a)b = a \times b$; r_b is the CB relative to the CG

`body.js:64-67, 95-138 · M is rebuilt only when σ, an image factor or the flooded mass changes by more than 0.2 %`

Coriolis, centripetal and Munk terms

A body that moves and rotates at once feels forces that do no work but redirect momentum. For the water this is the **Munk moment**: an elongated hull moving at an angle is turned broadside, which is why ROVs need yaw control at speed.

$$C_{RB}(\nu)\nu = \begin{bmatrix} m\,\omega \times v \\ \omega \times (I_g\,\omega) \end{bmatrix}, \qquad C_A(\nu_r)\nu_r = \begin{bmatrix} \omega \times p_t \\ v_r \times p_t + \omega \times p_r \end{bmatrix}$$

$$p_t = A_{11} v_r + A_{12}\,\omega, \qquad p_r = A_{21} v_r + A_{22}\,\omega \qquad \left(M_{A,e} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}\right)$$

`body.js:327-363 · this is Fossen's` $C_A = \begin{bmatrix} 0 & -S(p_t) \\ -S(p_t) & -S(p_r) \end{bmatrix}$

Damping

Fully coupled 6×6 linear and quadratic matrices. The linear part dominates at low speed, the quadratic part (form drag, $\frac{1}{2}\rho C_d A v|v|$) at higher speed. Damping fades as the vehicle leaves the water, and grows near the surface where it makes waves.

$$\tau_{D,i} = -s_d\,\beta_i \sum_{j=1}^{6}\Big(D_{L,ij}\,\nu_{r,j} + D_{Q,ij}\,|\nu_{r,j}|\,\nu_{r,j}\Big), \qquad s_d = \max(\sigma, 0.02), \quad \beta_i = \begin{cases} 1 + 0.6\cdot 4\sigma(1-\sigma) & i \le 3 \\ 1 & i > 3 \end{cases}$$

`body.js:312-325 · surface wave-making factor 0.6 (environment.js:32)`

Buoyancy and gravity

Weight acts at the CG, so it never makes a moment. Buoyancy acts at the CB; when the CB is above the CG the pair makes the righting moment that keeps the vehicle level. Water gets denser with depth and the hull can compress, so buoyancy changes slightly with depth. At the surface the hull is cut into volume elements, each partly submerged under the local wave height.

$$f_G^b = mg\,R^\top e_3, \qquad b_s = \frac{\rho(p_g)}{\rho_0}\,\max(0.5,\ 1-\beta p_g)$$

$$\text{submerged: } f_B^b = -\rho_0 g V b_s\,R^\top e_3, \quad m_B = r_b \times f_B^b$$

$$\text{at the surface: } \phi_e = \operatorname{clamp}\left(\frac{d_e + h_{z,e}}{2h_{z,e}}, 0, 1\right), \ \ \sigma = \frac{\sum_e v_e\phi_e}{V}$$

p_g	gauge pressure at the CG depth; β hull compressibility [1/Pa]
$d_e,\ h_{z,e}$	depth of element e below the local wave surface, and its vertical half-height

`body.js:243-288 · natural roll period shown in Vehicle:` $T_{roll} = 2\pi\sqrt{(I_{xx}+A_{44})/(\rho g V\,GM)}$

A housing past its depth rating leaks through an orifice against the air inside it, compressed by Boyle's law; the flooded water adds weight and mass and wets the leak probe:

$$p_{in} = \frac{p_0 V_a}{V_a - V_f}, \qquad \dot{V}_f = 0.62\,A_{\text{leak}}\sqrt{\frac{2\,(p_{out}-p_{in})}{\rho_0}}$$

`body.js:195-213`

Water, pressure, waves and current

Density comes from temperature and salinity (Tanaka 2001 for fresh water plus the UNESCO EOS-80 salt term), viscosity from Vogel's equation. Pressure is integrated down the water column with a compressible equation of state, so a Bar30 reads what it would in a real pool.

$$\frac{dp_g}{dz} = g\,\rho(p_g), \qquad \rho(p_g) = \frac{\rho_0}{1 - \bar{p}/K(S,T,\bar{p})}, \qquad p_{abs} = p_{atm} + p_g$$

`environment.js:141-174, water.js:19-43 · K is the EOS-80 secant bulk modulus`

Waves are linear (Airy) waves over finite depth. The wavenumber solves the dispersion relation by Newton's method; particle velocity and acceleration decay with depth.

$$\omega^2 = g\,k\tanh(kh), \qquad \eta = A\cos\varphi, \qquad \varphi = k(x\cos\beta + y\sin\beta) - \omega t$$

$$u = A\omega\frac{\cosh k(h-z)}{\sinh kh}\cos\varphi, \qquad w = -A\omega\frac{\sinh k(h-z)}{\sinh kh}\sin\varphi$$

$A,\ T_w,\ \beta$ wave amplitude, period $2\pi/\omega$, heading (Pool & water); off by default

`environment.js:101-236`

Accelerating water pushes the vehicle even when it is still: the Froude–Krylov force plus the added-mass reaction. Without it a neutral vehicle would not follow the waves.

$$\tau_{FK} = M_{A,e}[:,1{:}3]\;\dot{\nu}_c + \begin{bmatrix} F_{FK} \\ r_b \times F_{FK} \end{bmatrix}, \qquad F_{FK} = \rho_0 V b_s\, a_f^b, \qquad \dot{\nu}_c = a_f^b - \omega \times v_c^b$$

`body.js:294-309` · near the surface only the horizontal part is applied

Turbulence is a first-order Gauss–Markov gust, then smoothed so it can be differentiated:

$$x_{k+1} = a\,x_k + \sigma_t\sqrt{1-a^2}\;\xi_k, \qquad a = e^{-\Delta t/\tau}, \quad \xi \sim \mathcal{N}(0,1)$$

`core/rng.js:40-56, environment.js:177-188` · vertical intensity is half the horizontal

Floor and walls

Contact points are the extreme corners of the voxelised hull in 26 directions. Each one is tested against the floor (flat or sloped) and the four walls separately. A spring–damper pushes it out with Coulomb friction; the gains are capped from the vehicle's mass and the step size so the integration stays stable for a 2 kg or a 30 kg vehicle.

$$F_n = k\,\delta - c\,(V\cdot n) \quad (\delta > 0,\ F_n > 0), \qquad F_t = -\mu F_n \min\left(1, \frac{|v_t|}{v_{reg}}\right)\frac{v_t}{|v_t|}$$

$$k = \min\left(k_{user},\ m_{pt}\,\omega_c^2\right), \quad \omega_c = \frac{0.5}{\Delta t}, \qquad c \le \max\left(1.4\sqrt{k\,m_{pt}},\ 0.9\,\frac{m_{pt}}{\Delta t}\right)$$

`body.js:140-163, 390-433` · defaults k = 30 000 N/m, c = 900 N·s/m, μ = 0.5

Integration

Each 1.25 ms physics step: thrusters, battery and flooding are advanced once; then classic 4th-order Runge–Kutta integrates the state, split into more sub-steps when damping would make it stiff; the quaternion is renormalised after each.

$$y_{k+1} = y_k + \tfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \qquad q \leftarrow \frac{q}{\|q\|}$$

$$n = \min\left(64,\ \max\left(1,\ \left\lceil \tfrac{\lambda\,\Delta t}{2} \right\rceil\right)\right), \qquad \lambda = \max_i \frac{s_d\,(|D_{L,ii}| + 2|D_{Q,ii}\,\nu_i|)}{M_{ii}}, \qquad h = \frac{\Delta t}{n}$$

`body.js:465-504` · physics 800 Hz, firmware 400 Hz → 2 physics steps per firmware tick (simulation.js:80-84) · with damping off, energy is conserved to better than 10^{-4}

Where the code departs from textbook Fossen

- Added mass changes with submergence and wall distance but is frozen within a step; the $\dot{M}_A \nu$ term is not modelled.
- Quadratic damping uses the column form $D_{Q,ij}|\nu_j|\nu_j$ with a full matrix.
- A Froude–Krylov term is added for waves and gusts; the water itself does not rotate.
- The image effect uses distances from the CG along world axes.

CHAPTER 10 · HOW IT WORKS

10

Thrusters and battery

`src/physics/thruster.js` , `src/physics/battery.js`

PWM to thrust

A measured table (Blue Robotics T200 / T500 or your own) or a parametric curve gives static thrust from the PWM pulse. Inside the deadband thrust is exactly zero. A motor reversed with `MOT_n_DIRECTION` = −1 uses the curve mirrored about 1500 µs, so the strong side of the propeller stays the strong side.

$$T_0(P) = \begin{cases} T_{fwd}\left(\dfrac{P - hi}{P_{max} - hi}\right)^{e} & P > hi \\ 0 & lo \le P \le hi \\ -T_{rev}\left(\dfrac{lo - P}{lo - P_{min}}\right)^{e} & P < lo \end{cases} \qquad lo, hi = 1500 \mp \delta, \qquad T_{reversed}(P) = T_0(3000 - P)$$

defaults $T_{fwd} = 51.5$ N, $T_{rev} = 40.2$ N, $\delta = 25$ µs, $e = 1.25$, $P \in [1100, 1900]$

`thruster.js:12-21, 73-97, 164-170`

Rotor speed, voltage and current

The propeller cannot change speed instantly. A normalised rotor speed n follows the command with a faster time constant when speeding up than when braking, and pauses at zero when it reverses (the ESC re-commutates). Thrust grows with $n|n|$. Thrust and current follow the battery voltage through the Blue Robotics tables at 10–20 V.

$$n_{cmd} = \operatorname{sgn}(T_{cmd})\sqrt{\frac{|T_{cmd}|}{T_{side}}}, \qquad n_{k+1} = n_k + (n_{cmd} - n_k)\left(1 - e^{-\Delta t/\tau}\right), \qquad T = T_{side}\, n|n|$$

$$T_{cmd} = T_0(P)\; s_V(V_{bus}), \qquad s_V = \frac{\mathcal{I}(T_{max}, V)}{\mathcal{I}(T_{max}, V_{nom})}, \qquad I = I_0 + I_{max}(V)\left[\frac{|T|}{T_{side}\, s_V}\right]^{1.5}$$

T200 @ 16 V $\tau_{up} = 0.05$ s, $\tau_{down} = 0.075$ s, reversal pause 0.03 s; full thrust 2.98–6.70 kgf forward and 13.6–32 A between 10 and 20 V; the ESC cuts out below 6.5 V

`thruster.js:29-32, 141-197`

In-flow, ventilation, hull and reaction torque

Thrust falls when water already flows through the propeller (advance ratio), near the surface where the propeller sucks air, and when the jet hits the hull. The spinning propeller also twists the vehicle; mirror-image thrusters get opposite hands so a correctly built vehicle cancels it.

$$T_e = T \underbrace{\operatorname{clamp}\left(1 - \frac{V_{in}}{V_0}, 0, 1.25\right)}_{\text{in-flow}} \underbrace{\min\left(1, \sqrt{\tfrac{d}{1.5D}}\right)}_{\text{ventilation}} \underbrace{(1 - t_d)}_{\text{jet on hull}}, \qquad V_0 = J_0 \, D \, \frac{\text{rpm}_{max}}{60} \max(|n|, 0.05)$$

$$Q = -h \, \frac{K_Q}{K_T} \, D \, T, \qquad \frac{K_Q}{K_T} = 0.12, \qquad \tau_{thr} = \sum_k \begin{bmatrix} T_{e,k} \, a_k \\ r_k \times T_{e,k} \, a_k + Q_k \, a_k \end{bmatrix}$$

t_d	jet–hull deduction from 7 rays cast along the jet, up to 0.5 (forward and reverse separately)
J_0	0.9 forward, 0.75 reverse for the T200

`thruster.js:204-229, body.js:365-388, model.js:58-80`

Battery

A Thevenin model: open-circuit voltage from state of charge, an internal resistance that rises when the pack is nearly empty, and one RC branch for the slow voltage recovery after a burst. Default: 4S Li-ion, 18 Ah, 0.03 Ω, plus 2 A for the electronics.

$$V_t = n_{cells} \, \text{OCV}(\text{SoC}) - I \, R(\text{SoC}) - V_1, \qquad V_{1,k+1} = V_{1,k} \, e^{-\Delta t/\tau_1} + I R_1 \left(1 - e^{-\Delta t/\tau_1}\right)$$

$$\text{SoC} = 1 - \frac{\int I \, dt}{3.6 \, C_{mAh}}, \qquad R = R_0 \left[1 + 0.6 \, \frac{\max(0, 0.3 - s)}{0.3} + 2.5 \, \frac{\max(0, 0.05 - s)}{0.05}\right], \qquad R_1 = \tfrac{1}{2} R_0, \; \tau_1 = 30 \text{ s}$$

`battery.js:11-81 · at full forward the DUBURI sample reads 15.9–16.0 V at 24–27 A`

CHAPTER 11 · HOW IT WORKS

11

Added mass and damping

Where the matrices M_A, D_L and D_Q come from. AUTO-TUNE runs both methods at several resolutions and reports an uncertainty for each value.

Mass properties

The hull is voxelised (cell ≈ largest size / 56, at least 3 mm). Displaced volume and CB are integrated exactly along rays in x, y and z, weighted by how well each axis sees the surface; thin 12.7 mm plates come out within 0.04 %. Inertia uses uniform density and the parallel-axis theorem about the CG, unless you enter CAD values.

$$I_g = \rho_m I^u + m\left(|d|^2 I_3 - d\,d^\top\right), \qquad \rho_m = \frac{m}{V_{vox}}, \qquad M_G = H^\top M_O\, H, \quad H = \begin{bmatrix} I_3 & -S(r) \\ 0 & I_3 \end{bmatrix}$$

`model.js:41-56, bem.js:317-327 · H moves any 6×6 matrix from a reference point to the CG`

Panel method (BEM) for added mass

A Hess–Smith constant-source panel method on a smooth mesh of the hull, for a deeply submerged body in ideal flow. Six problems, one per DOF, share one influence matrix and are solved with GMRES; then the solution is extrapolated from three mesh sizes (900, 1800, 3600 panels).

$$\tfrac{1}{2}\sigma_i + \sum_{k\neq i} K_{ik}\,\sigma_k = N_j(x_i), \qquad K_{ik} = \frac{1}{4\pi}\int_{S_k} \frac{n_i\cdot(x_i-\xi)}{|x_i-\xi|^3}\,dS, \qquad N = \begin{bmatrix} n \\ (r-r_{ref})\times n \end{bmatrix}$$

$$\phi_{l,i} = \sum_k G_{ik}\,\sigma_{l,k}, \quad G = -\frac{1}{4\pi|x-\xi|}, \qquad m_{lj} = -\rho\sum_i \phi_{l,i}\,N_j(x_i)\,A_i, \qquad M_A = \tfrac{1}{2}\left(m + m^\top\right)$$

`bem.js:26-200 · GMRES restart 60, tolerance 10⁻⁸ · matches Capytaine within 0.2–2 % and the exact ellipsoid within 2 %`

Before AUTO-TUNE runs, the default is a Lamb ellipsoid with the hull's proportions and volume, with strip-theory floors on the rotational terms, moved to the CG.

Lattice-Boltzmann CFD for damping

A virtual tow tank on the GPU. D3Q19 lattice, regularised BGK collision with a Smagorinsky turbulence model, Bouzidi interpolated walls (moving walls for rotation), and forces by momentum exchange.

$$g_i^{eq} = w_i\left[(\rho-1) + \rho\left(3\,c_i\cdot u + 4.5\,(c_i\cdot u)^2 - 1.5\,|u|^2\right)\right], \qquad g_i^{*} = g_i^{eq} + \left(1-\frac{1}{\tau}\right)4.5\,w_i\left(c_{i\alpha}c_{i\beta} - \tfrac{1}{3}\delta_{\alpha\beta}\right)\Pi^{neq}_{\alpha\beta}$$

$$\tau = \tfrac{1}{2}\left(\tau_0 + \sqrt{\tau_0^2 + 18\sqrt{2}\,C_s^2\,|\Pi^{neq}|/\rho}\right), \qquad \tau_0 = 3\nu_{lat} + \tfrac{1}{2}, \qquad C_s = 0.12$$

$$F = \sum_{\text{links}} \left[c_{\bar{i}}\,(g_{\bar{i}}^{*} + g_i + 2w_i) - u_w\,(g_{\bar{i}}^{*} - g_i)\right]$$

`cfd/d3q19.js, cpu.js:120-217, gpu.js · weights 1/3, 1/18, 1/36`

Each DOF is towed (or spun) at two Reynolds numbers, 500 and 2000, on up to three grids. The coefficients are fitted to $C(Re) = C_\infty + K/Re$ and scaled to your vehicle in water: C_∞ becomes quadratic damping, K linear damping. The fit is

projected onto a dissipative matrix so it can never add energy.

$$D_{Q,ij} = -\rho\, s_j^2\, L^{p_i}\, C_{\infty,i}, \qquad D_{L,ij} = -\rho\, s_j\, L^{p_i - 1}\, K_i\, \nu, \qquad p = 2\ (\text{force}),\ 3\ (\text{moment})$$

$$\text{GCI} = \frac{1.25\,\left|\frac{f_1 - f_2}{f_1}\right|}{r_{21}^{p} - 1}, \qquad f_{ext} = \frac{r_{21}^{p} f_1 - f_2}{r_{21}^{p} - 1} \qquad \text{(Celik et al. 2008)}$$

`study.js:400-541, core/convergence.js:27-50 · sphere drag at Re 100: Richardson 1.074 vs 1.088 reference (−1.2 %)`

Rotation runs use a frozen hull with a moving-wall condition, so rotational damping carries an extra 30 % model uncertainty.

CHAPTER 12 · HOW IT WORKS

12

Sensors

The firmware never sees the true state, only what these models report.
`src/ardusub/sensors.js`

SENSOR	RATE	MODEL
IMU (gyro, accelerometer)	1 kHz	Truth interpolated between ticks, plus white noise (0.0015 rad/s, 0.04 m/s² per sample), a turn-on bias (0.02 °/s) and a bias random walk (0.0005 °/s/√s); harmonic notches, then a 2-pole low-pass at `INS_GYRO_FILTER` / `INS_ACCEL_FILTER` (20 Hz)
Depth (Bar02 / Bar30)	50 Hz → 10 Hz	Absolute pressure at the sensor's mounting point, 2 mm noise, quantised (Bar30 0.2 mbar), 20 ms latency; averaged at 10 Hz and converted as AP_Baro does
Vertical estimator	400 Hz	3-state Kalman filter (height, climb rate, accel bias) standing in for EKF3's vertical channel
Compass	10 Hz	Earth field rotated into the body, plus motor-current interference and `COMPASS_MOT_*` compensation
Rangefinder	10 Hz	Ray cast down the body axis to floor and walls, $d = t + (0.005 + 0.002t)\eta$; status from `RNGFND1_MIN/MAX_CM`
Battery monitor	10 Hz	Pack voltage and current with noise, mAh counting, resistance estimate as AP_BattMonitor
Leak probe	3 Hz	Wet when flooded volume exceeds 1 mL; `LEAKn_LOGIC`, analog or digital, 3 s hold
Temperature	5 Hz	Enclosure: $\tau\dot{T} = T_{water} + QR_{th} - T, Q = 8\ \mathrm{W} + 0.03\lvert VI\rvert, \tau = 600\ \mathrm{s}$
DVL / USBL (optional)	10 Hz	Position with 1 cm noise, only when *Positioning system present* is ticked

Depth: the AP_Baro water formula

$$\bar{p} = \frac{1}{n}\sum p_m, \qquad d = \frac{\bar{p} - p_0}{9800 \cdot \texttt{BARO_SPEC_GRAV}}, \qquad p_0 \leftarrow \bar{p} \ \text{ while } d < 0$$

`sensors.js:367-393` · `AP_Baro.cpp:901-944, ArduSub/sensors.cpp:4-16` · the physics uses the real ρg, so BARO_SPEC_GRAV is your correction, as on the vehicle

Vertical estimator

A small Kalman filter propagates height and climb rate with the accelerometer, delayed by `EK3_HGT_DELAY` (60 ms), and fuses each 10 Hz depth reading through an innovation gate.

$$x = [z,\ v_z,\ b_a], \qquad F = \begin{bmatrix} 1 & \Delta t & -\frac{1}{2}\Delta t^2 \\ 0 & 1 & -\Delta t \\ 0 & 0 & 1 \end{bmatrix}, \qquad P \leftarrow FPF^\top + Q$$

$$y = h - z, \quad S = P_{00} + R^2, \quad \frac{y^2}{S\gamma^2} \le 1, \quad K = \frac{P_{:,0}}{S}, \quad x \leftarrow x + Ky, \quad P \leftarrow P - KP_{0,:}$$

R	0.1 m with a depth sensor (ArduSub forces this), 10 m without
γ	`EK3_HGT_I_GATE` /100 = 5

`sensors.js:548-619 · an output predictor with EK3_TAU_OUTPUT smooths what the controllers read`

Attitude: the estimate is the true attitude with a slow Gauss–Markov error (0.02°), the unlearned gyro bias, and a yaw drift when there is no compass or DVL. `AHRS_TRIM` is cancelled by an equal board misalignment, as on a calibrated vehicle.

CHAPTER 13 · HOW IT WORKS

13

ArduSub control

Ported from ArduSub 4.5.7 and compared line by line with the C++ (`libraries/AC_PID` , `AC_AttitudeControl` , `AP_Motors` , `ArduSub/`). Units follow ArduPilot: centidegrees and cm at the interfaces.

The control chain for one tick, from stick to propeller:

1. **Pilot input** (joystick.cpp): MANUAL_CONTROL x, y, z, r become RC channels 1–6, scaled by the pilot gain.
2. **Flight mode** turns them into targets: lean angles, a yaw rate, a climb rate, or a position.
3. **Attitude controller**: input shaping → quaternion error → angle P (square-root controller) → body-rate target.
4. **Rate PIDs**: body-rate error → roll, pitch, yaw demands.
5. **Depth controller**: position P → velocity PID → acceleration PID → throttle.
6. **Mixer** (AP_Motors6DOF): six demands × factor table → one output per motor → PWM.

PID controllers

`AC_PID` is used for the rate loops (`ATC_RAT_*`) and vertical acceleration (`PSC_ACCZ_*`). Target and error are low-pass filtered, the derivative is taken on the filtered error, and the integrator stops growing while the motors are saturated.

$$\bar{r}_k = \bar{r}_{k-1} + \alpha_T\,(r_k - \bar{r}_{k-1}), \qquad \bar{e}_k = \bar{e}_{k-1} + \alpha_E\,(\bar{r}_{k-1} - y_k - \bar{e}_{k-1}), \qquad \bar{d}_k = \bar{d}_{k-1} + \alpha_D\left(\frac{\bar{e}_k - \bar{e}_{k-1}}{\Delta t} - \bar{d}_{k-1}\right)$$

$$\alpha(\Delta t, f_c) = \frac{\Delta t}{\Delta t + \frac{1}{2\pi f_c}}, \qquad I_k = \mathrm{clamp}(I_{k-1} + K_i\,\bar{e}_k\,\Delta t,\ \pm I_{max}) \text{ unless saturated}$$

$$u_k = \mu\left(K_p\,\bar{e}_k + K_d\,\bar{d}_k\right) + I_k$$

f_c	`FLTT` , `FLTE` , `FLTD` cut-offs (Hz); 0 disables the filter
μ	slew-limiter gain reduction from `SMAX` (1 when SMAX = 0); `PDMX` caps $\lvert P + D \rvert$
defaults	roll/pitch P 0.135, I 0.09, D 0.0036, IMAX 0.444; yaw P 0.18, I 0.018, IMAX 0.222

`pid.js:75-136 · AC_PID.cpp:193-320 · in ArduSub 4.5.7 the rate loop does not add FF, so ATC_RAT_*_FF and D_FF have no effect`

The **SMAX slew limiter** watches how fast P+D changes. When the slew rate stays above SMAX it lowers the gain, which stops oscillation before it builds up:

$$\mu = \begin{cases} \dfrac{S}{S + 1.5\,(\rho_{mod} - S)} & \rho_{mod} > S \\ 1 & \text{otherwise} \end{cases} \qquad \rho = \text{peak slew rate of } (P + D) \text{ over 300 ms windows, filtered at 25 Hz}$$

`control.js:343-387 · SlewLimiter.cpp:29-135`

The **square-root controller** is ArduPilot's shaped P controller: linear near the target, then a square root so the commanded rate never asks for more than the allowed acceleration.

$$\dot{x}(e) = \begin{cases} K_p\, e & |e| \le \dfrac{a_{max}}{K_p^2} \\ \operatorname{sgn}(e)\sqrt{2a_{max}\left(|e| - \dfrac{a_{max}}{2K_p^2}\right)} & \text{otherwise} \end{cases} \qquad \dot{x} \leftarrow \operatorname{clamp}\left(\dot{x},\ \pm\frac{|e|}{\Delta t}\right)$$

`control.js:23-38` · `AP_Math/control.cpp:401-433`

Attitude controller

The target attitude is a quaternion q_T. Pilot inputs are shaped into a smooth feed-forward rate ω_t with time constant `ATC_INPUT_TC`, and q_T advances by $\omega_t \Delta t$ each tick. The error to the measured attitude q_B is split into a tilt part and a heading part, and each goes through the angle P.

$$\omega_d \leftarrow \omega_d + \operatorname{sqrt_ctrl}\left(\theta_e,\ \tfrac{1}{\tau_{in}},\ \alpha_{max},\ \Delta t\right), \qquad \omega_t^{+} = \operatorname{clamp}(\omega_d,\ \omega_t \pm \alpha_{max}\Delta t)$$

$$\omega_{c,i} = \operatorname{sqrt_ctrl}\left(e_i,\ K_{ang,i},\ \tfrac{1}{2}\alpha_i,\ \Delta t\right), \qquad \omega_b = \omega_c + \left(q_B^{-1} q_T\right)\omega_t, \qquad u_{roll} = \mathrm{PID}_R(\omega_{b,x}, \hat{\omega}_x)$$

$$\omega = \begin{bmatrix} \dot{\phi} - \sin\theta\,\dot{\psi} \\ \cos\phi\,\dot{\theta} + \sin\phi\cos\theta\,\dot{\psi} \\ -\sin\phi\,\dot{\theta} + \cos\theta\cos\phi\,\dot{\psi} \end{bmatrix} \qquad \text{(Euler rates to body rates)}$$

defaults	`ATC_ANG_*_P` 6, `ATC_ACCEL_*_MAX` 110 000 cd/s² (1100 °/s²), `ATC_INPUT_TC` 0.15 s, `ATC_RATE_Y_MAX` 180 °/s
blend	above 30° tilt error the feed-forward fades out, above 60° only the tilt is corrected

`attitude.js:235-519` · `AC_AttitudeControl.cpp:269-1012, AC_AttitudeControl_Sub.cpp:421-432`

Depth and position

`AC_PosControl` runs a cascade: position error → velocity target → acceleration target → throttle around the hover value. Pilot climb commands are jerk-limited by `PSC_JERK_Z` and the filter cut-offs.

$$v_T = \operatorname{sqrt_ctrl}\left(z_T - \hat{z},\ K_{p,z}\right) + v_d, \qquad a_T = \mathrm{PID}_{vz}(v_T, \hat{v}_z) + a_d, \qquad u_{thr} = \frac{\mathrm{PID}_{az}(a_T,\ a_{z,meas})}{1000} + h$$

$$\dot{z}_{pilot} = \begin{cases} V_{dn}\dfrac{c - (500 - \delta)}{500 - \delta} & c < 500 - \delta \\ V_{up}\dfrac{c - (500 + \delta)}{500 - \delta} & c > 500 + \delta \\ 0 & \text{otherwise} \end{cases} \qquad \delta = \texttt{THR_DZ} \cdot G$$

h	hover throttle `MOT_THST_HOVER`, forced to 0.5 at boot by ArduSub
defaults	`PSC_POSZ_P` 3, `PSC_VELZ_P` 8, `PSC_ACCZ_P` 0.5, `PSC_ACCZ_I` 0.1, `PILOT_SPEED_UP` 100 cm/s, `PILOT_ACCEL_Z` 100 cm/s²

`poscontrol.js:131-265` · `AC_PosControl.cpp:701-1010, ArduSub/Attitude.cpp:90-125`

The XY controller (Position Hold, Guided, Auto) works the same way in two dimensions. ArduSub turns its lean-angle output into **forward and lateral thrust** instead of tilting the vehicle:

$$\theta_T = \arctan\frac{-a_f}{100g}, \qquad \phi_T = \arctan\frac{a_r\cos\theta_T}{100g}, \qquad F = \frac{\operatorname{clamp}(-\theta_T, \pm A)}{A}, \quad L = \frac{\operatorname{clamp}(\phi_T, \pm A)}{A}$$

`poscontrol.js:299-306, sub.js:550-554` · `ArduSub/motors.cpp:171-184` · `A = ANGLE_MAX`

Motor mixer

Each motor has a row of six factors (roll, pitch, yaw, throttle, forward, lateral) from the `FRAME_CONFIG` table or from the automatic allocation. `MOT_n_DIRECTION` multiplies the result. For the Vectored 6DOF frame (the DUBURI file, `FRAME_CONFIG` = 2) the attitude-plus-throttle part and the yaw-plus-translation part are normalised separately:

$$\mathrm{rpt}_i = \rho\, r_i + \pi\, p_i + T\, t_i, \qquad \mathrm{yfl}_i = \gamma\, y_i + F\, f_i + L\, l_i, \qquad u_i = \mathrm{clamp}\left(d_i \left(\frac{\mathrm{rpt}_i}{m_1} + \frac{\mathrm{yfl}_i}{m_2} \right), \pm 1 \right)$$

$$m_1 = \max\left(1, \max_i |\mathrm{rpt}_i|\right), \quad m_2 = \max\left(1, \max_i |\mathrm{yfl}_i|\right), \qquad \mathrm{PWM}_i = 1500 + u_i \cdot 400 \;\; (\mathrm{MOT_PWM}\ 1100\text{–}1900)$$

`motors.js:306-436 · AP_Motors6DOF.cpp:233-559 · other frames use one clamp per motor; Vectored (FRAME_CONFIG 1) limits forward thrust by MOT_FV_CPLNG_K`

A spool state machine (shut down → ground idle → spooling up → unlimited) ramps the output over `MOT_SPOOL_TIME` (0.5 s) after arming. `MOT_BAT_CURR_MAX` scales every output down when the predicted current is too high. Motor m drives the servo output whose `SERVOn_FUNCTION` is 33 + m, and the ESC sees the previous tick's value, as on the Pixhawk.

Main loop and scheduler

Each 2.5 ms tick runs ArduSub's fast loop in the same order as `ArduSub.cpp`, then the scheduled tasks whose interval has passed. Two physics steps follow with the PWM held.

1. Read IMU, AHRS and inertial navigation (the sensor models).
2. Run the rate controller (not in Manual or Motor Detect).
3. Motors output.
4. EKF yaw-reset check, flight mode update, surface and bottom detector.
5. Scheduled tasks (table below).
6. Physics: two 1.25 ms steps of the plant and environment.

TASK	RATE	EVERY N TICKS	WHAT IT DOES
`gcs update_receive`	400 Hz	1	MANUAL_CONTROL and heartbeats from the simulated GCS
`fifty_hz_loop`	50 Hz	8	Pilot-input, crash, EKF and depth-sensor failsafes; RC read; joystick and servo outputs
`read_rangefinder`	20 Hz	20	Rangefinder and SurfTrak terrain offset
`update_batt_compass`	10 Hz	40	Battery monitor and failsafe, compass
`update_altitude`	10 Hz	40	Depth sensor update and health
`update_turn_counter`	10 Hz	40	Tether turn counter
`temperature`	5 Hz	80	Internal temperature
`three_hz_loop`	3 Hz	133	Leak, internal pressure and temperature, GCS and terrain failsafes
`one_hz_loop`	1 Hz	400	Pre-arm status, notch sample rate
`AP_Arming update`	1 Hz	400	PreArm messages every 30 s

The 3 Hz loop runs every 133 ticks, which is 3.008 Hz: AP_Scheduler computes it the same way. Telemetry is recorded at 50 Hz (12 minutes in memory).

CHAPTER 14 · HOW IT WORKS

14

AUTO-TUNE

`src/sim/tuning.js` . Every test runs on a fresh copy of your vehicle in open water; results are suggestions and nothing is applied until you click.

1. **Hydrodynamics.** Converged panel-method added mass and CFD damping for your hull, each with its uncertainty.
2. **Rate loops**, inner loop first, following ArduPilot's AC_AutoTune: twitch the axis, raise D until the rate bounces back more than 10 % (AGGR 0.1), then raise P until the rate overshoots, then bisect between good and bad.
3. **Angle P** by angle twitches of 20° (30° in yaw), backed off to 90 %; `ATC_ACCEL_*_MAX` from the measured acceleration.
4. **`ATC_INPUT_TC`** : kept unless the tuned loops overshoot more than 10 % with it.
5. **Depth** (accel → velocity → position) and, with a DVL, **position** gains by a grid search refined with golden-section steps on the step-response cost.
6. **Check**: each axis is compared with your current gains on the full tuned set; an axis that was better before keeps its gains.

$$J = 0.5\,T_s + 0.05\,\mathrm{OS} + 0.1\,\max(0, \mathrm{OS} - 10) + 0.4\,N_{osc} + 3\,\mathrm{sat} + 0.6\,T_r + 2\,\frac{\mathrm{RMS}_{tail}}{\mathrm{band}}$$

$$J_{eval} = 0.6\,J(\mathrm{step}) + 0.4\,J\left(-\tfrac{1}{2}\mathrm{step}\right)$$

T_r, T_s	10–90 % rise time and settling time (s); OS overshoot (%); N_{osc} oscillations; sat fraction of time the mixer saturated
margin	rate P and D are multiplied by 0.85, and every value is clamped to the ArduSub parameter range

`tuning.js:142-719 · AC_AutoTune_Multi.cpp constants`

On a well-damped ROV the D search can reach the top of its range without ever seeing a bounce; the panel marks such values as "at the limit".

CHAPTER 15 · REFERENCE

15

Parameters

All 2708 parameters ArduSub 4.5.7 defines, generated from the ArduPilot source by `tools/gen-params.mjs`, with the values from the DUBURI Pixhawk file.

- **Loading.** Mission Planner (`NAME,value`) and QGroundControl (`sysid compid NAME value type`) files are both read. Every line is kept; names ArduSub 4.5.7 does not define are kept too and written back unchanged.
- **Simulated or stored.** 671 parameters change what the simulator does (gains, mixer, failsafes, sensors, joystick, outputs). The other 2037 (calibration, serial ports, logging, EKF tuning…) are stored and exported exactly.
- **Validation.** Edits are clamped to the ArduPilot range; values loaded from a file are never changed. `MOT_THST_HOVER` is forced to 0.5 at boot, as ArduSub does.
- **Legacy names** are converted on import: `ATC_RAT_*_FILT` → `FLTE`, `FS_BATT_*` → `BATT_*`, `MNT_` → `MNT1_`, `CAM_` → `CAM1_`.
- **Export** writes the 943 names of your file plus any you changed, with full float precision; the QGC format carries the right MAVLink type per parameter.

Values the DUBURI file changes from the ArduSub defaults

The full searchable list of all 2708 parameters is in the online documentation: jarifalam13.github.io/DUBURI_SIM/guide.

PARAMETER	MEANING	DEFAULT	DUBURI
AHRS_TRIM_X	AHRS Trim Roll	0	0.006551251
AHRS_TRIM_Y	AHRS Trim Pitch	0	0.02238178
BARO_PRIMARY	Primary barometer	0	1
BARO1_DEVID	Baro ID	0	721674
BARO1_GND_PRESS	Ground Pressure	0	100825.5
BARO2_DEVID	Baro ID2	0	1209865
BARO2_GND_PRESS	Ground Pressure	0	72209.78
BRD_TYPE	Board type	0	2
BTN0_SFUNCTION	Function for button when the shift mode is toggled on	0	8
BTN1_SFUNCTION	Function for button when the shift mode is toggled on	0	12
BTN2_SFUNCTION	Function for button when the shift mode is toggled on	0	63
BTN3_SFUNCTION	Function for button when the shift mode is toggled on	0	64
BTN4_SFUNCTION	Function for button when the shift mode is toggled on	0	53
BTN5_SFUNCTION	Function for button when the shift mode is toggled on	0	1
COMPASS_DEV_ID	Compass device id	0	131594

PARAMETER	MEANING	DEFAULT	DUBURI
COMPASS_OFS_X	Compass offsets in milligauss on the X axis	0	306.1081
COMPASS_OFS_Y	Compass offsets in milligauss on the Y axis	0	-58.63057
COMPASS_OFS_Z	Compass offsets in milligauss on the Z axis	0	160.0112
COMPASS_PRIO1_ID	Compass device id with 1st order priority	0	131594
COMPASS_USE	Use compass for yaw	1	0
COMPASS_USE2	Compass2 used for yaw	1	0
EK3_ALT_M_NSE	Altitude measurement noise (m)	2	0.1
EK3_GBIAS_P_NSE	Rate gyro bias stability (rad/s/s)	0.001	0.0001
EK3_GYRO_P_NSE	Rate gyro noise (rad/s)	0.015	0.025
EK3_IMU_MASK	Bitmask of active IMUs	3	1
EK3_MAG_CAL	Magnetometer default fusion mode	3	2
EK3_SRC1_YAW	Yaw Source	1	0
FORMAT_VERSION	Eeprom format version number	0	1
FRAME_CONFIG	Frame configuration	1	2
FS_GCS_ENABLE	Ground Station Failsafe Enable	2	0
FS_LEAK_ENABLE	Leak Failsafe Enable	1	0
FS_PILOT_INPUT	Pilot input failsafe action	2	0
INS_ACC_ID	Accelerometer ID	0	1246218
INS_ACC1_CALTEMP	Calibration temperature for 1st accelerometer	-300	55.17224
INS_ACC2_CALTEMP	Calibration temperature for 2nd accelerometer	-300	53.11743
INS_ACC2_ID	Accelerometer2 ID	0	1114634
INS_ACC2OFFS_X	Accelerometer2 offsets of X axis	0	0.711316
INS_ACC2OFFS_Y	Accelerometer2 offsets of Y axis	0	1.002211
INS_ACC2OFFS_Z	Accelerometer2 offsets of Z axis	0	0.8100671
INS_ACC2SCAL_X	Accelerometer2 scaling of X axis	1	1.040886
INS_ACC2SCAL_Y	Accelerometer2 scaling of Y axis	1	1.007392
INS_ACC2SCAL_Z	Accelerometer2 scaling of Z axis	1	1.001601
INS_ACCOFFS_X	Accelerometer offsets of X axis	0	0.1722613

PARAMETER	MEANING	DEFAULT	DUBURI
INS_ACCOFFS_Y	Accelerometer offsets of Y axis	0	-0.1976387
INS_ACCOFFS_Z	Accelerometer offsets of Z axis	0	-0.7587661
INS_ACCSCAL_X	Accelerometer scaling of X axis	1	0.9957629
INS_ACCSCAL_Y	Accelerometer scaling of Y axis	1	0.9964735
INS_ACCSCAL_Z	Accelerometer scaling of Z axis	1	0.9845021
INS_GYR_ID	Gyro ID	0	2163722
INS_GYR1_CALTEMP	Calibration temperature for 1st gyroscope	-300	62.39164
INS_GYR2_CALTEMP	Calibration temperature for 2nd gyroscope	-300	60.49401
INS_GYR2_ID	Gyro2 ID	0	2228490
INS_GYR2OFFS_X	Gyro2 offsets of X axis	0	0.03468281
INS_GYR2OFFS_Y	Gyro2 offsets of Y axis	0	-0.01690163
INS_GYR2OFFS_Z	Gyro2 offsets of Z axis	0	-0.002528002
INS_GYROFFS_X	Gyro offsets of X axis	0	0.02375078
INS_GYROFFS_Y	Gyro offsets of Y axis	0	-0.0078316
INS_GYROFFS_Z	Gyro offsets of Z axis	0	-0.02835965
MNT1_TYPE	Mount Type	1	0
MOT_2_DIRECTION	Motor normal or reverse	1	-1
MOT_3_DIRECTION	Motor normal or reverse	1	-1
MOT_7_DIRECTION	Motor normal or reverse	1	-1
MOT_8_DIRECTION	Motor normal or reverse	1	-1
SERVO1_FUNCTION	Servo output function	0	33
SERVO2_FUNCTION	Servo output function	0	34
SERVO3_FUNCTION	Servo output function	0	35
SERVO4_FUNCTION	Servo output function	0	36
SERVO5_FUNCTION	Servo output function	0	37
SERVO6_FUNCTION	Servo output function	0	38
SERVO7_FUNCTION	Servo output function	0	39
SERVO8_FUNCTION	Servo output function	0	40

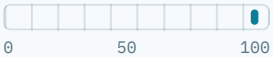
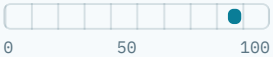
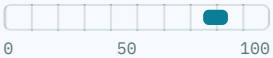
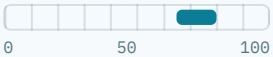
PARAMETER	MEANING	DEFAULT	DUBURI
SERVO9_FUNCTION	Servo output function	59	**181**
STAT_BOOTCNT	Boot Count	0	**762**
STAT_FLTTIME	Total FlightTime	0	**160737**
STAT_RUNTIME	Total RunTime	0	**732328**
TEMP1_ADDR	Temperature sensor address	0	**119**
TEMP1_BUS	Temperature sensor bus	0	**1**
TEMP1_TYPE	Temperature Sensor Type	0	**1**

CHAPTER 16 · REFERENCE

16

Accuracy and limits

Engineering estimates of how closely the real vehicle will behave like the simulation, assuming you uploaded your own design with measured mass and CG. They have not been checked against pool data yet.

WHAT	ESTIMATED MATCH	0 – 100 %	WHY
Controller decisions: mode logic, failsafes, PWM per thruster	~95 %		Ported line by line from ArduSub 4.5.7 and tested; not the firmware binary, simpler estimator than EKF3.
Stability: does a gain set oscillate, does the vehicle tip	85–90 %		Set mostly by mass, CG, CB and thruster layout, which match well when measured.
Depth hold: overshoot and settling time	75–85 %		Buoyancy and thrust are modelled well; damping carries uncertainty.
Speed, acceleration and turning	65–80 %		Drag comes from CFD (±20–30 %) and the tether is not modelled.

In short: the **kind** of behaviour (which way it moves, whether a gain set oscillates, what each mode does) matches about 90 %; the **numbers** (speed, overshoot, settling time) about 70–80 %. Gains tuned here are in the right range but may need 10–30 % adjustment in the pool.

Not modelled

- The tether. A real tether adds drag and pulls the vehicle, most in sway and at depth.
- Wake interaction between thrusters, windmilling drag of a stopped propeller, wave radiation damping.
- EKF3 itself (a simpler estimator with realistic noise stands in), GPS, and a real MAVLink connection.
- Temperature changing with depth; thrusters and electronics changing with pressure.

Make it trustworthy

Weigh the vehicle, find the CG with a hang test, and, if you can, measure thrust on a stand. Then do three simple pool runs (full forward, full up, a yaw turn) and compare the logs with the same runs here.

CHAPTER 17 · REFERENCE

17

Presenting the simulator

A one-minute pitch, a five-minute demo and the questions a supervisor is likely to ask.

One-minute pitch

"Sir, the DUBURI simulator lets us test the robot on a computer before it goes in the water. It uses our own 3D design and the exact parameter file that is on our Pixhawk."

"Inside, the ArduSub 4.5.7 control code is ported line by line and checked against the ArduPilot source. The motion is computed with the Fossen 6-DOF equations that marine robotics uses."

"So we can check the thruster layout, the PID gains and depth hold before a pool session. That saves pool time and battery, and lowers the risk of damaging the vehicle."

"The numbers won't match exactly. Our estimate is about 90 % for the kind of behaviour and 70–80 % for numbers like speed and overshoot. One measured pool test would let us calibrate it."

Five-minute demo

1. Open the simulator and click **Fly the sample first**.
2. Press `Enter` to arm, fly with `W` `A` `S` `D` and `R` / `F`.
3. Press `3` (Depth Hold), let go of the keys, press `G`: the vehicle keeps its depth and the graphs show target and actual.
4. Open **Frame** and move the Surge slider: show which thrusters fire and the force that comes out.
5. Open **Params**: our 943 Pixhawk parameters are loaded; change a gain and the effect is immediate.
6. Open **Tuning** and run a yaw step test: rise time, overshoot and settling time appear.
7. Set `FS_GCS_ENABLE` to 3, click **GCS link**: about 2.5 s later the vehicle switches to Depth Hold on its own.
8. Upload our design (OBJ) and show the thrusters being found automatically.

AUTO-TUNE runs the panel method and CFD and can take several minutes; start it before the meeting if you want to show its result.

Likely questions

Is this the real ArduSub firmware? –

No. It is a JavaScript port of the ArduSub 4.5.7 control code (motors, attitude, PID, depth and position control, modes, failsafes, joystick), compared line by line with the C++ source. It is not the firmware binary (that would be SITL), and EKF3 is replaced by a simpler estimator.

How close is it to the real robot? –

About 90 % for the kind of behaviour and 70–80 % for the numbers, as an estimate. The largest uncertainties are drag (CFD, ±20–30 %), the missing tether and the mass and CG we enter. See Accuracy and limits.

Why not ArduPilot SITL with Gazebo? –

SITL runs the real firmware and is a good cross-check, but it needs a Linux setup and a separately built physics model. DUBURI runs in any browser, builds the hydrodynamic model from our own CAD, and loads our Pixhawk file directly. Both can be used together.

What do we need to measure? –

Mass in air, centre of gravity, ideally thrust per thruster, and one short pool test to compare with the simulation.

Can the exported parameters harm the vehicle? –

The export is an ordinary `.param` file with your original values plus the ones you changed. Review the changed values in QGroundControl before writing them, as with any parameter change.

CHAPTER 18 · REFERENCE

18

Troubleshooting

Common messages and fixes

"Compass performance degraded" every time I arm

The DUBURI file has `GPS_TYPE` = 1 and there is no EKF origin, so ArduSub prints this warning. Real ArduSub does the same. Set `GPS_TYPE` = 0 if the vehicle has no GPS.

"Mode change failed: ... requires position"

Position Hold, Surface, Guided, Auto and Circle need a position estimate. Tick **Positioning system present (DVL / USBL)** in Pool & water, only if the real vehicle has one.

"Mode change failed: ... need alt estimate"

Depth modes need a working depth sensor. Check the depth sensor in Pool & water and `BARO_EXT_BUS` / `BARO_PROBE_EXT`.

The vehicle will not arm

Read the PreArm banner on the HUD: it gives ArduSub's own reason. Arming also needs a joystick button mapped to arm or arm-toggle, and is refused while the GCS link is down.

The gamepad is not detected

Press any button once. Use Chrome or Edge. Some embedded views hide controllers: save the page from Logs & files and open the file directly.

The 3D view is blank

The simulator needs WebGL2. Turn on hardware acceleration in the browser settings, or try Chrome or Edge.

A thruster was detected that is not a thruster

Short hollow parts 85–115 mm across look like a T200 duct. Discard the detection or rename the part so it contains "mount" or "housing".

CHAPTER 19 · REFERENCE

19

Development

Plain ES modules, built into one HTML file with esbuild. GPL-3.0-or-later, because the control code is a port of ArduPilot.

```
npm install
npm test               # 185 unit tests
npm run build          # dist/duburi-sim.html, docs/index.html (GitHub Pages)
npm run build:wasm     # recompile src/wasm/kernels.c (clang with wasm32)
npm run validate:cfd   # sphere drag through the full CFD pipeline
node tools/gen-params.mjs   # regenerate the parameter table (needs an ArduPilot checkout)
```

FOLDER	CONTENTS
`src/ardusub/`	ArduSub 4.5.7 port: params, motors, attitude, PID, position control, navigation, joystick, RC, sensors, modes and failsafes
`src/physics/`	6-DOF plant, thrusters, battery, environment (pool, water, waves, current)
`src/hydro/`	Water properties (EOS-80), panel method (BEM), hull meshing
`src/cfd/`	D3Q19 lattice-Boltzmann on CPU and WebGL2, tow-tank studies
`src/geometry/`	STL/OBJ/PLY, voxelisation, mass properties, thruster detection
`src/alloc/`	ArduSub frame tables, thruster allocation and DOF analysis
`src/sim/`	Simulation loop, telemetry, step tests and AUTO-TUNE
`src/ui/`	three.js scene, HUD, panels, graphs

Every difference found between the earlier port and ArduSub 4.5.7, and how it was fixed, is listed in AUDIT.md (109 findings).